Cybersecurity Exam Study Module

Units 1 and 2
From the basics to exam answers

Based on your syllabus, 248 teaching pages and slides, two study documents, and all 16 files in the past question archive, six additional paper screenshots, and all 41 pages of your official class notes.

PKI infrastructure is excluded.
Mathematical support and extra slide material are clearly distinguished from the syllabus core.

Includes 37 worked paper and class questions and illustrations, 32 retrieval questions, a 50 mark practice paper, labeled algorithm flows, and a complete source and scope audit.

Contents

0 How to use this module 3
1 Cybersecurity and the CIA triad 6
2 Threats attack surfaces risk and measurement 9
3 Incident analysis for the exam 13
4 Cryptography and the three data states 15
5 Mathematical tools from scratch 17
6 Keys randomness and computational security 22
7 Symmetric asymmetric and hybrid cryptography 27
8 DES AES and cipher design 30
9 Block cipher modes and their failure patterns 34
10 Hash functions MD5 SHA 256 and the birthday bound 37
11 MACs digital signatures and non repudiation 40
12 RSA from key generation to correctness 43
13 Diffie Hellman key agreement 47
14 Authentication and strong authentication 49
15 Authorization access control and privilege 51
16 Put the mechanisms together 54
17 Extensions from the supplied slides and older papers 55
18 Solved questions from your archive and new paper 61
19 Revision and practice 70
20 What not to memorize incorrectly 74
21 Audit of every supplied file and past paper 75
22 Corrections and source notes 83

0 How to use this module

This module teaches **Units 1 and 2 of PC IT CSE301 from the beginning**, using your syllabus, all six slide decks including the PDF deck, both study documents, and every file in the past question ZIP, plus the six subsequently supplied paper screenshots and all 41 pages of the official class notes, cs_notesmidsem.pdf. It combines explanation, worked calculations, diagrams to reproduce, exam answer patterns, and practice. The aim is to understand the material well enough to explain it, calculate with it, and apply it to unfamiliar scenarios.

Scope decision. Your request supplies Unit 1 and Unit 2 material, so the examinable core here is those two units. The syllabus also lists Units 3 and 4, but this module does not turn into a full application or network security course. **PKI infrastructure is excluded as requested.** Digital signatures remain included. Because the syllabus names digital certificates alongside PKI, section 11.8 retains a brief certificate concept and format overview without the infrastructure, trust chains, or revocation procedures. If your teacher's exclusion covers that entire combined line, you can skip section 11.8 too.

Core means explicitly listed in Units 1 or 2. **Supporting** means slide material or mathematical machinery needed to answer the relevant papers properly. **Extension** means material from your slides or older papers beyond the written syllabus. Learn the core first, then supporting calculations and diagrams, then the extension if you have time. A topic appearing in an old paper does not automatically make it part of your current syllabus.

0.1 The learning route

Read Chapters 1 to 4 to learn what security protects and what cryptography does. Study Chapters 5 to 7 before doing RSA and DH calculations. Work through Chapters 8 to 13 for the algorithms and their security properties. Chapters 14 and 15 cover authentication and permissions. Chapter 16 connects the pieces. Chapter 17 contains the extra slide material. Chapter 18 solves selected actual past questions. Chapters 19 to 22 provide revision, practice, the full archive audit, and corrections to misleading source statements.

After each chapter, close the notes and answer its checkpoint. If you cannot answer in your own words, return to the explanation before memorizing a model answer. For every numerical problem, write the formula, substitute values, show reductions, and check the result. The older papers explicitly require major calculation steps, sometimes without a calculator.

0.2 Syllabus coverage map

Syllabus requirement	Where taught	Required ability
Cybersecurity and CIA	Chapters 1 and 3	Define and analyze each objective separately
Threats to CIA and attack surfaces	Chapter 2	Identify weakness, attack path, impact, and control
Incidents and high level analysis	Chapter 3	Explain cause, CIA effects, response, and prevention
Cryptography for rest motion and process	Chapter 4	Match state to exposure and protection
Symmetric and asymmetric methods together	Chapters 7 and 16	Explain hybrid protection and key distribution
DES and AES outlines	Chapter 8	Draw structures and explain components
RSA and DH outlines	Chapters 12 and 13	Calculate, explain, and prove correctness
Integrity and non repudiation	Chapters 10 and 11	Choose a hash, MAC, or signature correctly
MD5 and SHA 256 outlines	Chapter 10	Explain construction and compare properties
Proper pseudorandom generation	Chapter 6	Explain entropy, seeds, unpredictability, and failure
Strong authentication	Chapter 14	Distinguish factors and stop guessing or replay
Authorization and access control	Chapter 15	Separate policy from enforcement
Mandatory discretionary and role based models	Chapter 15	Compare decision authority and use cases
Privilege and privilege escalation	Chapter 15	Distinguish vertical and horizontal violations
Digital certificate concept	Brief supporting section 11.8	Understand a signed identity to public key binding if retained by the teacher
PKI infrastructure	Excluded	No trust chain issuance or revocation study required here

0.3 What the past papers tell us

Highest priority for repeated practice: RSA setup, encryption and signing, Euler's theorem, modular inverses, correctness versus security, hash properties, DES structure, symmetric versus asymmetric cryptography, and DH steps and correctness. The evidence includes the 2023 C1 paper, Quiz_20, the older midsemester papers, and the 2023 C3 DH question.

High priority for scenario answers: CIA analysis, active and passive attacks, insiders, password search space, authentication, authorization, and broken access control. The September 2025 midsemester paper and its supplied answer sheet use Fortinet and Equifax scenarios. The 2025 answer collection includes login brute force and unauthorized profile changes.

Supporting material worth learning: group and field vocabulary, repeated squaring, block modes, HMAC, the birthday effect, nonces, and the limits of textbook RSA. These appear in the supplied slides and relevant older questions.

Extension rather than automatic core: ElGamal, ECC point arithmetic, RC4, random oracle proofs, and post quantum lattices. **Outside this module's core:** Kerberos, named zero knowledge protocols, BAN logic, PGP packet formats, SSL handshake details, IPSec, WEP, Snort rules, scanning commands, injection code, and firewall configuration. Chapter 21 audits them so you know what to skip for a Unit 1 and 2 exam.

New paper evidence: the printed 18 September 2025 midsemester paper explicitly asks ROM, group orders, ECC subtraction, and elliptic curve ElGamal. These remain supporting or extension topics under the written syllabus, but their direct appearance in a recent midsemester paper makes them worth preparing after the core. Chapters 6, 11, 17, and 18 now include the missing variants.

These are priorities supported by this archive, not predictions of the next paper. The C1 paper occurs twice in different scans and counts as one distinct paper. Some filenames disagree with the printed year; the audit uses the printed identity whenever readable.

0.4 How the official class notes change your preparation

The official notes reinforce the mathematical preparation needed for RSA and DH: Fermat and Euler, inverses, repeated squaring, groups, reductions, and correctness proofs. They also teach ECC, modular square roots, polynomial fields, ROM, and quantum security. These receive explicit class examples here, while remaining marked supporting or extension wherever they exceed the written syllabus. Do not skip their mathematics merely because it is less prominent in the slides; prepare it after the named core algorithms.

New class exercises are solved in sections 18.30 to 18.37. Section 21.17 accounts for every official-notes page, including repeated photographs and unrelated material. Chapter 22 identifies corrections to handwritten simplifications. The timing scribbles on page 39 are revision cues, not a confirmed duration or marking scheme for your upcoming examination. Your stated syllabus and PKI exclusion still govern this module.

1 Cybersecurity and the CIA triad

Core. Main sources: Introduction to Cyber Security, PDF pages 4 to 18 and 39 to 45; the syllabus Unit 1; Introduction to Cryptography slides 5 to 13.

1.1 Begin with a university portal

A portal stores names, grades, fees, and login details. Students need to see their own records. Teachers need to enter grades. Administrators maintain the service. Security must allow these useful actions while resisting unauthorized actions and accidental failures. A system that denies everyone access may keep records secret, but it fails its purpose.

Cybersecurity is the protection of digital systems, networks, services, and their information against unauthorized access, misuse, alteration, disruption, and other threats. It includes people, procedures, and technology. **Information security** also protects information outside digital systems, such as printed records and spoken confidential information. The areas overlap.

An **asset** is something valuable: information, a server, an account, a service, reputation, or the ability to operate. Decide what is valuable before choosing a control. A hospital's patient data and its ability to deliver care are both assets.

1.2 Confidentiality

Confidentiality means information is disclosed only to authorized parties. A student seeing another student's private marks violates confidentiality. So do intercepted plaintext messages, exposed database backups, stolen unencrypted drives, and excessive employee access.

Encryption can protect data even if an attacker obtains the storage medium or network traffic. Access restrictions reduce who may read the data. Neither works alone in every situation: an authorized application may decrypt a database and then expose its contents through an incorrect permission check.

Typical controls: encryption, least privilege, access control, careful sharing, and physical protection. Confidentiality also includes sensitive metadata in some contexts. Encrypting message text does not necessarily hide who contacted whom or how much data they sent.

1.3 Integrity

Integrity means data and system state remain accurate and complete, and unauthorized changes can be prevented or detected. Changing a fee payment from 500 to 50000, altering a grade, deleting an audit entry, or installing a modified update can violate integrity.

Data integrity concerns the content. **Origin authenticity** concerns who produced it. A message may be unchanged after receipt yet falsely attributed to someone else. These ideas are connected, but they answer different questions.

A hash detects a difference only when the expected hash is trustworthy. A MAC or signature can bind the content to a secret key or signing key. These mechanisms generally **detect** tampering when verification occurs. Permissions and secure workflows help prevent unauthorized writes. Database transactions and backups also support integrity.

1.4 Availability

Availability means authorized users can obtain the service and data when needed, at an acceptable level of performance. A crashed exam portal, a ransomware encrypted hospital database, an overloaded login service, and a lost encryption key can each deny availability.

Typical controls: capacity planning, redundancy, tested backups, disaster recovery, rate limiting, resilient network design, monitoring, and incident response. Encryption alone does not stop traffic flooding or hardware failure.

Availability can be measured, for example, as uptime divided by the total observation time. An uptime target is meaningful only if you specify the service and observation period. A login system may technically run while taking ten minutes to respond; latency is also part of useful availability.

1.5 Analyze each objective separately

Scenario	Confidentiality	Integrity	Availability
Attacker reads a database without changing it	Violated	No alteration established	No outage established
Attacker changes grades	Possibly affected if records were also read	Violated	May remain available
Traffic flood blocks the portal	No disclosure established	No alteration established	Violated
Ransomware encrypts files	Potentially affected if theft also occurs	Unauthorized change affects integrity	Violated until recovery
Powered off encrypted laptop stolen	May be preserved if keys remain protected	No data change established	Lost access to that device or copy

Exam method: state what the scenario proves, then what it makes possible. Super administrator access creates a confidentiality risk, but access capability alone is not evidence that every file was stolen. An answer that labels all three as compromised without an explanation is weaker than a careful causal analysis.

1.6 The seven requirements in the crypto slides

The slides extend CIA with authentication, non repudiation, authorization, and access control. **Authentication** verifies an identity or origin claim. **Non repudiation** provides evidence of an action that can be checked by others. **Authorization** decides permitted actions. **Access control** enforces those permissions.

For the portal: a login verifies a student's account claim; a policy permits reading that student's marks; the server checks the request and enforces that policy. A signed grade approval can provide evidence of the teacher's action. None of these replaces service availability.

1.7 People process and product

People: users, administrators, developers, security teams, and attackers. Training helps staff recognize suspicious messages and report mistakes quickly. Usable controls matter because confused users often bypass difficult workflows.

Process: access approval, patch management, backup restoration, incident escalation, change review, and accountability. A written rule without enforcement and review is ineffective.

Product or technology: encryption, secure software, firewalls, monitoring, and endpoint protection. A tool depends on configuration and operation. A backup product does not prove recovery is possible until someone tests restoration.

Defense in depth uses complementary layers. For a phishing attempt: awareness may prevent the click; email filtering may block the message; MFA may reduce account takeover; least privilege may limit damage; monitoring may detect misuse; recovery procedures may restore the service. Design layers assuming one can fail.

1.8 Governing principles from the slides

- **No absolute security:** practical protection reduces risk within assumptions and resources. Compromised endpoints or leaked keys can defeat strong cryptography.
- **Defense in depth:** combine controls across different failure points.
- **Complexity increases risk:** more interfaces, dependencies, and exceptions make mistakes harder to detect. Simpler configurations are easier to review.
- **Risk management:** prioritize what could happen and how serious it would be.
- **Measurement:** use defined indicators to judge effectiveness, rather than confidence alone.
- **Continuous improvement:** plan a control, implement it, check results, and revise it.
- **Human limitations:** design for errors and deception rather than depending on perfect behavior.
- **Evolving threats:** review protections as attacker capabilities and systems change.

The slide phrase that humans are the weakest link is a useful warning about social engineering, not a claim that technology and process are always stronger. Security failures often involve all three.

1.9 Checkpoint and exam answer pattern

Checkpoint: An attacker changes a student's recovery email, downloads the marks, and locks the student out. Which actions affect C, I, and A? **Answer:** downloading affects confidentiality; changing the email affects integrity; lockout affects availability. The permission flaw and authentication consequence should also be identified.

For a short CIA question, give three definitions and three matched examples. For a scenario question, describe the actual action, identify the objective, and link a relevant control to that action. Avoid merely listing product names.

2 Threats attack surfaces risk and measurement

Core with slide support. Main source: Introduction to Cyber Security pages 19 to 38 and 44 to 52; C1 2023 questions 8 to 10.

2.1 The vocabulary chain

A **threat** is a potential cause of harm. A **vulnerability** is a weakness that may permit that harm. An **attack** is an attempt to exploit a weakness or otherwise defeat a protection. An **exploit** is the method or code used to take advantage of a vulnerability. A **control** reduces the likelihood or consequences. **Risk** is the combination of likelihood and impact in a given context.

Example: the asset is a student database. The threat is an attacker seeking private records. The vulnerability is a server missing a security patch. The attack uses the flaw to obtain access. The impact is disclosure. Controls include patching, restricting exposure, segmentation, monitoring, and limiting database privileges.

A threat is possible even if no attack has happened. A vulnerability is not automatically exploitable under every deployment. A control can reduce risk without eliminating the vulnerability, such as restricting an exposed interface until the patch can be applied.

2.2 Attack surface and attack vector

The **attack surface** is the collection of reachable points and interactions through which a system could be attacked. The **attack vector** is a particular route used in an attack. A public login page belongs to the surface; credential stuffing through that page is a vector.

Surfaces include public websites and APIs, administrative interfaces, user accounts, email, mobile devices, Wi Fi, exposed cloud storage, physical access, software dependencies, and vendor connections. Internal services also matter after an attacker gets an initial foothold.

To reduce the surface: remove unused services, restrict management access, patch reachable applications, review permissions, limit third party access, and inventory dependencies. An inventory is necessary because an unknown service rarely receives deliberate protection.

2.3 Passive and active attacks

Passive attacks observe without deliberately modifying the communication or service. Eavesdropping threatens content confidentiality. Traffic analysis studies timing, volume, and relationships even when content is encrypted. Passive attacks can be difficult to detect because observation may leave little trace.

Active attacks change behavior, data, or service. Examples are message modification, masquerading as another user, replaying a recorded request, redirecting communication, and denial of service. Active attacks may affect integrity, authenticity, availability, and confidentiality together.

Replay is active even if the message bytes are unchanged: the attacker reuses a valid message in a new context. A recorded transfer approval must not authorize the same transfer again. Fresh challenges, sequence numbers, and transaction binding address this problem.

2.4 Threat categories linked to CIA

Threat	What happens	Primary objective at risk	Relevant defenses
Eavesdropping	Reads traffic	Confidentiality	Authenticated encryption in transit
Unauthorized database access	Reads protected records	Confidentiality	Server permission checks and least privilege
Tampering	Changes data or settings	Integrity	MACs or signatures and write restrictions
Masquerade	Uses false or stolen identity	Authentication and associated permissions	Strong authentication and credential protection
Replay	Reuses an old valid request	Freshness and integrity of actions	Nonces sequence numbers and expiry
DoS or DDoS	Exhausts resources	Availability	Rate limits filtering resilience and capacity
Ransomware	Encrypts or blocks data and may steal it	Availability integrity and possibly confidentiality	Patch management segmentation and tested backups
Supply chain compromise	Abuses a trusted dependency or update	Potentially all CIA properties	Supplier review update verification and least privilege

These defenses are conceptual Unit 1 knowledge. Detailed Snort rules, firewall syntax, and application exploits belong to later units.

2.5 Adversaries and insider advantage

Cybercriminals often seek money through fraud, ransom, or resale. State actors may seek strategic intelligence or disruption. Hacktivists may seek publicity for a cause. Insiders may act maliciously or make mistakes. Opportunistic attackers may use existing tools without understanding their details.

An insider may already possess credentials, network access, knowledge of valuable systems, familiarity with routines, and awareness of weak controls. This can bypass the initial access effort an outsider faces. An insider's actions may initially resemble legitimate use. Controls include least privilege, separation of duties, logging, and review of unusual privileged activity.

An outsider who steals an employee account can acquire some insider advantages. The useful distinction is the access and trust available, not merely where the attacker sits geographically.

2.6 Breach anatomy and the kill chain

The slides summarize a breach as **infiltration, propagation, aggregation, exfiltration**. Infiltration is entry. Propagation is movement to additional systems. Aggregation is collection of useful information. Exfiltration is its transfer out. A ransomware attack may also disrupt operations.

The seven step **Cyber Kill Chain** gives another high level model:

1. **Reconnaissance:** gather information about targets and exposed systems.
2. **Weaponization:** prepare the malicious payload or attack package.

3. **Delivery:** send or place it through an available route.
4. **Exploitation:** trigger the weakness to obtain execution or access.
5. **Installation:** establish malware or persistence.
6. **Command and control:** communicate with the compromised environment.
7. **Actions on objectives:** steal, alter, disrupt, or extort.

It is a reasoning framework. Real attacks may skip, repeat, or reorder stages. A credential theft attack may never install malware.

Worked example: an attacker researches a university, prepares a fake account notice, sends it to staff, steals credentials through a counterfeit login, enters the portal, creates persistent access, then downloads records. Email controls target delivery; MFA targets account misuse; server authorization limits what the account reaches; monitoring targets unusual activity; incident response revokes sessions and repairs damage.

2.7 Risk management

Identify assets and dependencies. Identify threats and weaknesses. Estimate likelihood and impact. Select controls. Reassess the remaining **residual risk**. For example, an exposed administrator portal with a known exploited vulnerability is more urgent than an isolated test system with no sensitive information.

The expression **risk approximately equals likelihood times impact** is a planning model, not a physical law. A score is meaningful only if the scales and assumptions are clear. A vulnerability severity score also differs from organizational risk: exposure, data sensitivity, and existing controls affect the latter.

Responses include reducing risk with controls, avoiding a risky activity, transferring some consequences through contracts or insurance, and explicitly accepting a remaining risk. Insurance does not patch the system or remove the duty to protect it.

2.8 Metrics from the slides

Mean time to detect or MTTD is the average delay between incident occurrence and detection. **Mean time to respond, contain, restore, or recover** has several possible meanings under the abbreviation MTTR. Define the endpoint before calculating it. Your slides use hours to contain after detection.

For detection delays 2, 5, 1, 8, and 4 hours, MTTD = 20/5 = **4 hours**. For post detection containment delays 3, 1, 4, 2, and 5 hours, the mean containment delay = 15/5 = **3 hours**. The mean occurrence to containment interval for these same incidents is 7 hours.

Other measures include patch delay, percentage of required systems tested, MFA coverage, restoration test results, incident count, and incident cost. A **leading indicator** suggests future exposure, such as overdue patches. A **lagging indicator** records an outcome, such as completed incidents.

Zero detected incidents can mean strong defenses or weak detection. Faster ticket closure can mean efficient response or premature closure. Compare periods only when definitions and coverage remain consistent. Use measurements to make a decision, not merely to display an attractive number.

2.9 Supporting slide concepts

The Gen I to Gen V sequence in the slides is an illustrative history from standalone viruses to network, application, targeted payload, and broad multivector attacks. These generations overlap; firewalls did not make viruses disappear. Memorize the progression only after understanding the threats and controls.

Zero trust removes automatic trust based solely on being inside a network. Evaluate identity, device and request context, limit privileges, and contain access. It is an architectural approach, not a single product.

AI may assist phishing, impersonation, anomaly detection, and alert analysis. Growing dependency chains make vendor compromise important. Quantum computing affects cryptographic assumptions, covered in Chapter 17. The regulation and career slides give context, but exact legal obligations and dated market statistics are not necessary to answer the Unit 1 syllabus objectives.

2.10 Checkpoint

Question: A staff member receives a fake email, enters a password, and the attacker exports records. Identify surface, vector, weakness, attacker action, and control. **Answer:** email and login interfaces are surfaces; phishing is the vector; susceptibility to the deception plus inadequate account protection are weaknesses; stolen credential use and exfiltration are actions; awareness, phishing resistant authentication, least privilege, and monitoring interrupt different stages.

3 Incident analysis for the exam

Core. Main evidence: CS mid sem ans_pdf.pdf and the incident and threat slides. Learn the reasoning, rather than a fixed list of historical names.

3.1 A repeatable analysis method

Start with **the asset and service**. State **the entry point and weakness**. Explain **the attacker's resulting capability**. Analyze **C, I, and A separately**. Give **immediate containment**, then **recovery**, then **long term prevention**. Distinguish confirmed facts from plausible impacts.

For a three mark CIA question, make three short paragraphs or rows. Each should say the objective, the concrete effect, and why the scenario supports that effect. For a longer question, add the process failures and specific controls.

3.2 Fortinet authentication bypass scenario

The supplied midsemester answer sheet describes the January 2025 exploitation of **CVE 2024 55591** in FortiOS and FortiProxy. An authentication bypass permitted super administrator access. The vulnerability identity and exploitation listing are supported by the NIST vulnerability record.

Asset and cause: a network security device and its management plane; an authentication check can be bypassed through a vulnerable path. The device's role makes administrator compromise especially serious because it controls traffic and policy.

Confidentiality: privileged access can expose configuration, logs, credentials, and information available through the device. State disclosure as a capability or risk unless the question states confirmed theft.

Integrity: the attacker can change security rules, accounts, and settings, or interfere with logs. Unauthorized configuration changes directly threaten integrity.

Availability: the attacker could disable services, lock out administrators, or create disruptive policies. In the supplied scenario availability is potentially affected; an actual outage must be supported by facts in the question.

Immediate actions: identify affected devices, follow the vendor's patch or mitigation guidance, restrict exposed management access, investigate unusual privileged accounts and changes, and preserve evidence. A patch closes the weakness but does not remove an already created attacker account.

Long term actions: maintain an accurate device inventory, monitor vendor advisories, test emergency patch procedures, minimize management exposure, review privileged access, and detect unexpected configuration changes. Separate administration from general user traffic where practical.

3.3 Equifax 2017 scenario

The supplied answer sheet uses the Equifax breach and **CVE 2017 5638**, an Apache Struts remote code execution vulnerability associated with multipart request handling. It describes roughly 147 million affected people. The GAO investigation documents the breach and weaknesses in identification, detection, access segmentation, and data governance. Counts vary with the investigation date; the exact number is secondary to the causal analysis.

Cause: an internet facing application remained vulnerable. A crafted request could cause unauthorized execution. This is a patch management and exposure problem as well as a software flaw.

Confidentiality: unauthorized access and extraction of personal data establishes disclosure. **Integrity:** execution capability creates the possibility of altering records and application state, but do not claim a specific alteration without evidence. **Availability:** service interruption is possible from compromised execution, but it is not the main demonstrated outcome in the supplied account.

A remotely reachable attack, no required authenticated account, low attack complexity, and broad consequences explain a high technical severity assessment. These are better reasons than simply saying the company was large.

Immediate response to discovering exposure: identify affected versions and assets, restrict or temporarily isolate the vulnerable service where feasible, apply a fixed release, preserve and inspect logs, investigate compromise, and use temporary request filtering only as an interim defense. Then review the patch rollout and verification failure, limit application database access, and improve detection.

3.4 Ransomware example

The May 2017 WannaCry campaign illustrates how an unpatched network service can enable broad disruption. The historical campaign and SMB link are described in the CERT EU advisory.

The useful Unit 1 lesson is the relationship between an exposed weakness, propagation, and availability loss. Encrypting files without authorization changes their state and denies normal use. Modern ransomware can also steal data, but theft should not be assumed for every historical event.

Controls act at different stages: patches reduce initial exploitation; segmentation limits spread; least privilege limits reachable files; tested offline or protected backups aid recovery; monitoring and response reduce time before containment. Backups alone do not prevent confidential data theft.

3.5 Practice scenario

Question: A university leaves an old administration interface publicly reachable. Attackers exploit it, create an administrator account, change grades, and disable the portal. Give a high level analysis.

Answer: the attack surface includes the exposed administration interface; the vulnerability is the exploitable outdated software. Administrator account creation gives persistence and excessive privilege. Reading records is a confidentiality risk, grade alteration violates integrity, and portal disablement violates availability. Isolate the exposed interface, patch, remove unauthorized accounts, revoke sessions, verify grades against trustworthy records, restore service, and review privileged logs. Long term controls include inventory, patch verification, restricted management access, least privilege, and monitoring.

4 Cryptography and the three data states

Core. Sources: Introduction to Cryptography slides 3 to 18 and both identically worded study documents.

4.1 What cryptography actually does

Cryptography designs mathematical mechanisms for security properties such as secrecy, integrity, and authentication under stated assumptions. **Encryption** converts plaintext into ciphertext using a key. **Decryption** uses the appropriate key to recover plaintext. **Cryptanalysis** studies how protections can fail. A **protocol** combines algorithms, messages, checks, and assumptions to accomplish a larger task.

Write symmetric encryption as C = E_K(M) and decryption as M = D_K(C). Correctness requires D_K(E_K(M)) = M. An attacker may know the algorithm and see ciphertext while lacking K.

Kerckhoffs's principle: the mechanism should remain secure when its design is public and only the secret key remains unknown. Public designs enable examination and interoperability. Hiding implementation details may add inconvenience for an attacker, but it should not be the foundation of secrecy.

4.2 Choose the mechanism by objective

Mechanism	Main contribution	Does not automatically provide
Encryption	Confidentiality	Sender identity integrity or availability
Unkeyed cryptographic hash	Compact fingerprint for comparison	Trusted origin or protection when attacker replaces hash too
Message authentication code	Integrity and authenticity among key holders	Public proof of which holder generated it
Digital signature	Integrity public verification and evidence of origin	Message secrecy
Authentication mechanism	Confidence in identity or key possession	Permission for every requested action
Access control	Enforcement of permission	Encryption of leaked storage or traffic

An encryption algorithm can be part of authenticated encryption, but the added authentication mechanism supplies the integrity protection. Do not credit bare encryption with all security services.

4.3 Data at rest

Data at rest sits on disks, database files, USB drives, backups, or cloud storage. Threats include stolen media, improper permissions, exposed storage, and copied backups. Full disk encryption protects against reading a drive without its key. File, database, or column encryption offers different levels of protection.

Example: a powered off encrypted laptop is stolen. If the key remains protected, possession of the drive need not reveal its contents. If malware runs after the legitimate user unlocks the disk, the malware may read plaintext through the user's session. Encryption at rest protects a particular boundary, not every possible access path.

Keys must be stored separately from the material they protect or behind a suitable security boundary. A backup containing ciphertext and an unprotected decryption key gives the thief both pieces. Losing every copy of the key may make data permanently unavailable.

4.4 Data in motion

Data in motion moves between systems. Threats include sniffing, message modification, impersonation, and replay. Authenticated encrypted channels, VPNs, and end to end messaging protect different paths.

With a properly established encrypted channel, a packet capture can record ciphertext and metadata without revealing application content. It may reveal IP addresses, timing, volume, and some protocol information. Access to endpoint secrets or a compromised endpoint changes the threat model.

The difference between **transport encryption** and **end to end encryption** is which systems can decrypt. A connection to a service may protect the network path while the service still sees plaintext. End to end protection aims to keep intermediaries from reading the content.

TLS is used as a familiar example here. Detailed handshake messages, certificate validation, and PKI are outside this module's required study scope.

4.5 Data in process

Data in process is information actively used in RAM, CPU operations, or application computations. Ordinary encrypted storage generally gets decrypted before processing. A key or plaintext may therefore exist in memory.

Trusted execution environments aim to isolate computations and memory from parts of the surrounding system. Their assurance depends on hardware and implementation assumptions. **Homomorphic encryption** permits certain calculations on ciphertext, so a processor can operate without learning the input plaintext. **Secure multiparty computation** allows parties to compute a joint result while limiting disclosure of their inputs.

For the syllabus, understand the problem and the outline of these approaches. You do not need to derive homomorphic algorithms. The key point is that disk and network encryption alone do not protect ordinary plaintext processing from a compromised application.

4.6 Cryptography cannot solve every problem

Encryption does not repair incorrect authorization, prevent users from sending secrets to a fraudster, keep a powered off server running, or undo a leaked key. A signature does not prove a document is factually true. A hash does not establish who published a file unless its reference value has a trustworthy origin.

Checkpoint: A company encrypts its database files but allows any logged in user to download every employee's salary. Is confidentiality protected? **Answer:** encryption helps against raw disk access; the application permission failure still exposes salary plaintext. The system needs correct access control as well as encryption.

5 Mathematical tools from scratch

Supporting and essential for calculations. Sources: Preliminaries slides 13 to 27; RSA and DH slides; Mid-Sem_16.pdf printed 2018 questions 1 and 2; Quiz_20 questions 4 and 7.

5.1 Bits bytes XOR and hexadecimal

A bit is 0 or 1. Eight bits form one byte. A 128 bit AES block contains 16 bytes. Hexadecimal uses digits 0 to 9 and A to F, with one digit representing four bits. For example, 0xAA = 10101010 in binary.

XOR, written XOR or the symbol in your slides, outputs 1 when its two input bits differ and 0 when they agree. Thus 0 XOR 0 = 0, 0 XOR 1 = 1, 1 XOR 0 = 1, and 1 XOR 1 = 0. It applies position by position to strings: 0110 XOR 1010 = 1100.

The crucial property is (M XOR K) XOR K = M. The second XOR cancels the first. This explains stream encryption and much of block cipher key mixing. XOR alone with a repeated predictable mask is weak; security comes from how the mask or transformation is generated.

Rotation moves bits cyclically, with bits leaving one end returning at the other. A left rotation by two changes 10110010 into 11001010. A shift instead discards exiting bits and fills vacant positions. Rotation is reversible; a shift generally loses information.

5.2 Modular arithmetic

The expression a mod n means the remainder represented between 0 and n minus 1. For example, 17 mod 5 = 2. Congruence a = b mod n means n divides a minus b. Think of a clock: after the last residue, counting wraps around.

You can reduce during addition and multiplication:

```
(a + b) mod n = ((a mod n) + (b mod n)) mod n
(a * b) mod n = ((a mod n) * (b mod n)) mod n
```

This prevents huge intermediate numbers. Negative residues are valid shorthand: minus 2 mod 7 = 5. If 23 is equivalent to minus 10 mod 33, then 23 squared is equivalent to 100 mod 33 = 1.

5.3 GCD and modular inverses

The **greatest common divisor**, gcd(a,n), is the largest positive integer dividing both. Numbers are **coprime** when the gcd is 1. Coprime does not mean both numbers are prime: 8 and 15 are coprime.

A multiplicative inverse a^(-1) mod n is a number b satisfying ab mod n = 1. It exists exactly when gcd(a,n) = 1. For example, 5^(-1) mod 13 = 8 because 5 times 8 = 40 = 1 mod 13. In contrast, 2 has no inverse mod 8 because 2 times any integer is even and cannot give a remainder of 1.

Modular division means multiply by an inverse. The expression 3/5 mod 7 means 3 times 5^(-1) mod 7. Since 5^(-1) = 3, the answer is 9 mod 7 = 2. Ordinary decimal division does not apply.

5.4 Euclid and extended Euclid

Euclid repeatedly divides with remainder until the remainder is zero. For gcd(35,192): 192 = 5 times 35 + 17; 35 = 2 times 17 + 1; 17 = 17 times 1. Therefore gcd = 1.

Extended Euclid works backward to express 1 as a combination of the original numbers:

```
1 = 35 - 2*17
  = 35 - 2*(192 - 5*35)
  = 11*35 - 2*192
```

So 35 times 11 = 1 mod 192, and the inverse of 35 mod 192 is **11**. This is how you compute RSA's private exponent. Always check by multiplying e times d and reducing modulo the totient.

5.5 Repeated squaring

To compute a large modular power, square and reduce instead of multiplying a by itself many times. The exponent's binary expansion tells you which powers to combine.

Example: 5^13 mod 23. Since 13 = 8 + 4 + 1:

```
5^1 mod 23 = 5
5^2 mod 23 = 25 mod 23 = 2
5^4 mod 23 = 2^2 mod 23 = 4
5^8 mod 23 = 4^2 mod 23 = 16
5^13 mod 23 = 16*4*5 mod 23 = 320 mod 23 = 21
```

There are approximately log base 2 of the exponent many squarings, rather than a number proportional to the exponent. This makes the forward operations of RSA and DH efficient even for large values. The arithmetic cost of multiplying large integers still matters.

5.6 Euler's totient and theorem

Euler's totient phi(n) counts residues from 1 to n minus 1 that are coprime to n. For a prime p, phi(p) = p minus 1. For distinct primes p and q, phi(pq) = (p minus 1)(q minus 1).

Euler's theorem: if gcd(a,n) = 1, then a^phi(n) = 1 mod n. A useful consequence is a^(k*phi(n)+1) = a mod n for nonnegative integers k. These are often called the two versions in your older papers. State the coprimality condition.

Example: phi(15) = 8. Since gcd(7,15) = 1, 7^8 = 1 mod 15, so 7^7 is the inverse of 7. In fact, 7 times 13 = 91 = 1 mod 15, giving inverse 13.

Why the theorem works: multiplying every invertible residue by a coprime a permutes the invertible residue set. The product of the new set is therefore equal to the product of the original set modulo n. The new product contains a factor a^phi(n). Cancel the product of the invertible residues using its inverse to obtain the theorem.

For noncoprime a, the theorem may fail. For example, 2^phi(8) = 2^4 = 0 mod 8, not 1. RSA correctness for noncoprime messages needs an additional argument in Chapter 12.

5.6.1 Fermat's little theorem and choosing an inverse method

Official notes pages 7, 8, 17, 18, 27, and 28. Fermat's little theorem says that for prime p and a not divisible by p, a^(p-1) = 1 mod p. For example, 3^6 = 1 mod 7. Euler generalizes this to a possibly composite modulus, provided gcd(a,n) = 1: replace p-1 by phi(n). Fermat's little theorem is different from Fermat's last theorem about sums of powers.

The equivalent form **a^p = a mod p holds for every integer a**, including multiples of p. For a multiple of p both sides are zero; otherwise multiply the first form by a. You may reduce an exponent modulo p-1 when a is nonzero modulo p. Do not reduce blindly when the base is zero modulo p: replacing a positive exponent by zero would incorrectly introduce a^0 = 1.

To find an inverse, choose among three methods:

- **Extended Euclid:** works for any modulus n > 1 when gcd(a,n) = 1; it does not require factoring n. Usually the most general hand method.

- **Fermat:** for prime p and nonzero a, a^(-1) = a^(p-2) mod p. Compute the power by repeated squaring.

- **Euler:** for gcd(a,n) = 1, a^(-1) = a^(phi(n)-1) mod n. Useful when the totient is already known; computing it for an unknown large RSA modulus is not a free step.

Why the power formulas work: a times a^(phi(n)-1) equals a^phi(n), which is 1. Thus the power is exactly a multiplicative inverse. Fermat is the prime special case. For tiny examples, trial multiplication is also legitimate, but it scales poorly. Section 18.33 solves the exact inverse examples from class using powers and verifies them.

Totient calculations beyond pq. For a prime power p^r, phi(p^r) = p^r - p^(r-1), since exactly the multiples of p are excluded. If gcd(u,v) = 1, phi(uv) = phi(u)phi(v). Therefore, using the distinct prime divisors of n, phi(n) = n times the product of (1 - 1/p). For n = 12 = 2^2 * 3, phi(12) = 12*(1/2)*(2/3) = 4. The units modulo 10 are 1, 3, 7, and 9, giving phi(10) = 4; 5 is excluded because gcd(5,10) = 5.

5.6.2 Chinese remainder theorem from a small example

Supporting tool, mentioned in official notes pages 10 and 20. If moduli are pairwise coprime, a specified residue for each determines a unique residue modulo their product. Solve x = 2 mod 3 and x = 3 mod 5. The candidates from the first condition are 2, 5, 8, 11, 14 modulo 15; only **8** also has remainder 3 modulo 5. Every solution is 8 + 15k.

An algebraic solution is x = 2 + 3t. The second condition gives 3t = 1 mod 5. Since 3^(-1) = 2 mod 5, t = 2 mod 5, so x = 8 mod 15. This converts simultaneous conditions into one congruence using an inverse.

Why RSA uses this idea: if two integers agree modulo p and modulo q for distinct primes, their difference is divisible by both and hence by pq. This completes RSA correctness for all message representatives. For square roots modulo pq with known factors, find roots modulo p and q, then combine root pairs. Section 18.34 illustrates four roots; the product of coprime moduli is essential to the uniqueness claim.

5.7 Groups rings and fields

A **group** is a set with an operation that has closure, associativity, an identity, and an inverse for every element. Closure means operating on elements gives another element in the set. Associativity means regrouping does not change the result. An identity leaves every element unchanged. An inverse undoes an element's operation. If order also does not matter, the group is **Abelian**.

For integers mod 6 under addition, the elements are 0 to 5, the identity is 0, and the inverse of 4 is 2 because 4 + 2 = 0 mod 6. Addition is commutative. Under multiplication, all residues mod 6 do not form a group: 0 and 2 lack multiplicative inverses.

The **units modulo n**, denoted Z_n*, are exactly the residues coprime to n. These form a multiplicative group. For prime p, every nonzero residue is a unit. DH uses a suitable cyclic group, often a carefully chosen subgroup of the nonzero residues modulo a prime.

A **ring** has addition and multiplication: an Abelian group under addition, associative multiplication, and distributivity. Multiplicative inverses are not required. Z_8 is a ring but has nonzero elements without inverses.

A **field** is a commutative ring with an identity in which every nonzero element has a multiplicative inverse. Z_p is a field when p is prime. Z_7 is a field; Z_8 is not. Rational numbers form an infinite field. A finite field has finitely many elements.

Important correction to the slides: RSA arithmetic modulo n = pq takes place in a ring, with its invertible elements forming a group. Z_n for composite RSA n is not a field. DH can use a multiplicative subgroup over a prime field. AES uses GF(2^8).

Connecting the definitions. A ring has an Abelian addition group, but its multiplication need not be commutative or give every element an inverse. A field additionally has commutative multiplication, an identity 1 distinct from 0, and inverses for every nonzero element. Its nonzero elements therefore form an Abelian multiplication group. Zero has an additive inverse, namely zero, but never a multiplicative inverse in a field.

For Z_8, 2*4 = 0 although neither factor is zero. Such factors are **zero divisors**, and this cannot occur in a field: if ab = 0 and a is nonzero, multiply by a^(-1) to obtain b = 0. Do not confuse Z_10, which includes every residue and is a ring, with Z_10* = {1,3,7,9}, which is its group of units under multiplication. The star here means units; it does not mean exponentiation.

5.8 Cyclic groups and generators

A group is cyclic if one element generates all its elements by repeated application of the operation. For a multiplicative group, a generator g produces powers 1, g, g^2, and so on. Its **order** is the smallest positive r with g^r = 1.

In Z_7*, 3 produces 3, 2, 6, 4, 5, 1, so it has order 6 and generates all nonzero residues. Knowing the group and generator is not secret in DH. The private exponent is secret.

Official notes page 5 example: the powers of 3 modulo 11 are 3, 9, 5, 4, 1. Thus ord(3) = 5. The set {1,3,9,5,4} is a subgroup of Z_11*, but 3 does not generate all ten units. In contrast, the page 16 powers of 2 are 2, 4, 8, 5, 10, 9, 7, 3, 6, 1. Consequently 2 has order 10 and is a generator, and log base 2 of 9 modulo 11 is **6**.

A **subgroup** is a subset that itself forms a group using the same operation. The order of an element divides the size of a finite group. Every cyclic group is Abelian, but an Abelian group need not be cyclic. For example, every nonidentity element of Z_8* = {1,3,5,7} squares to 1, so no single element generates all four. Likewise, an entire elliptic curve group need not be cyclic; cryptographic protocols select a suitable cyclic subgroup.

5.9 GF 2 to the power 8

GF(2^8) contains 256 elements represented as bytes. It is not the same arithmetic as integers modulo 256: the latter has noninvertible even elements. Interpret byte bits as coefficients of a polynomial over GF(2), then perform polynomial arithmetic modulo an irreducible degree eight polynomial.

For AES, the polynomial is x^8 + x^4 + x^3 + x + 1. Addition becomes XOR because coefficients are mod 2. Every nonzero field element has an inverse. AES uses these operations in SubBytes and MixColumns. Learn this interpretation for the AES outline. The official class notes also include a polynomial calculation, so the small supporting example below is worth practicing; implementing full AES arithmetic is beyond this module.

5.9.1 Polynomial field arithmetic from the official notes

Official notes page 38; supporting class calculation. GF(p^r) is a field with p^r elements, where p is prime. GF(2) supplies the bit coefficients; GF(2^8) supplies AES bytes. For r > 1 this is not integer arithmetic modulo p^r.

Start with a smaller field GF(2^3). Represent each element using a polynomial of degree less than 3 with bit coefficients. Choose f(x) = x^3 + x + 1 as the irreducible modulus. Irreducible means it cannot be factored into nonconstant lower degree polynomials over GF(2). This cubic has no root at 0 or 1, so it is irreducible. The reduction rule is **x^3 = x + 1**, because addition and subtraction coincide for bit coefficients.

Addition is coefficientwise XOR: (x^2+x) + (x+1) = x^2+1, represented by 110 XOR 011 = 101. Multiplication first expands the polynomials and then removes multiples of f(x). For example:

```
x^2 * (x+1) = x^3 + x^2
              = x^2 + x + 1 mod f(x)
Binary: 100 * 011 = 111 in this field
```

The multiplication sign here means field multiplication, not ordinary binary integer multiplication. The class's larger input 1010 represents x^3+x and must first be reduced to the canonical three bit representation 001. Its multiplication by 0011 gives 011. Section 18.36 shows both reduction routes so the four bit input does not obscure the field size.

An inverse: x*(x^2+1) = x^3+x = 1 modulo f(x), so 010 and 101 are multiplicative inverses. Thus dividing by x means multiplying by x^2+1. In general, polynomial extended Euclid finds an inverse modulo f(x), just as integer extended Euclid finds one modulo n. Zero has no inverse.

AES uses a different modulus, x^8+x^4+x^3+x+1; do not use the cubic above in an AES answer. The official AES specification defines that byte field. The cubic is a small teaching model of the same kind of arithmetic.

5.10 Checkpoint

Find 1/5 mod 13 and 3/5 mod 7. **Answers:** 8 and 2. Explain why Z_8 is not a field. **Answer:** nonzero 2 has no multiplicative inverse. Explain why modular exponentiation is efficient. **Answer:** repeated squaring uses a number of modular multiplications proportional to the exponent's bit length.

6 Keys randomness and computational security

Core for PRNG; supporting for complexity. Sources: Preliminaries slides 3 to 12 and 22 to 32; Symmetric Cryptography slide 20; C1 2023 questions 1 and 9.

6.1 Keys and passwords

A **cryptographic key** is input controlling an algorithm, usually generated as high entropy bits. A symmetric key is secret among its intended holders. A public key can be shared, while its corresponding private key must remain protected.

A **password** is usually a human usable secret for authentication. Human choices are not uniformly random. Length alone does not tell you its entropy. A long predictable phrase can be easier to guess than a shorter randomly generated string.

If a password has L characters chosen independently and uniformly from an alphabet of size A, there are A^L candidates and L*log2(A) bits of entropy. For five lowercase letters or digits, the space is 36^5 = **60,466,176**, about **25.85 bits**. This is an upper bound for that format under uniform choice, not a reliable estimate for arbitrary human passwords.

6.2 Brute force and security lifetime

For a uniformly random b bit key, exhaustive search has 2^b candidates. A worst case search tries all candidates. The expected successful search takes about 2^(b-1) trials, assuming a single correct key and an identifiable success. At r trials per second:

```
Worst case time = number of candidates / r
Expected time   = number of candidates / (2*r)
```

For the five character example at one million trials per second, worst case time is about 60.47 seconds and expected time about 30.23 seconds. This illustrative rate must be stated. Online rate limits may make guesses far slower; offline guessing of a stolen verifier depends on the password hashing cost and hardware.

Key lesson: a 256 bit output derived from a 26 bit secret does not acquire 256 bits of unpredictable input. More output bits do not manufacture entropy. A secure design uses appropriate randomness and a mechanism that raises the cost of guesses.

6.3 PRNG TRNG and CSPRNG

A **PRNG** expands a seed deterministically. The same seed and state produce the same sequence. This is useful for simulations, but statistical appearance alone does not ensure security. An attacker may infer the state or predict future outputs.

A **TRNG or entropy source** samples physical phenomena. Its output still needs assessment and conditioning: physical sources can be biased, fail, or be influenced. Physical origin alone does not prove high entropy.

A **cryptographically secure PRNG or CSPRNG** aims to produce output computationally unpredictable to an attacker without its protected state. A practical construction combines entropy inputs with a secure deterministic generator. Some designs also provide resistance to learning past outputs after current state compromise and recover future secrecy after fresh entropy is added; these properties depend on the design.

For keys, use the operating system's cryptographic random facility rather than timestamps, predictable counters, or simulation generators. The required property is attacker unpredictability, not merely a balanced count of zeroes and ones.

6.4 Why randomness matters everywhere

Weak random generation can make AES keys guessable, RSA primes repeat across devices, DH private exponents predictable, and authentication challenges replayable. The mathematical algorithm does not repair a predictable secret.

Random generation also supports RSA encryption padding and some signing schemes. Certain signatures can leak private keys when required per signature randomness is reused or biased. Learn the general failure: an assumption about unpredictable secret inputs has been violated.

6.5 Key nonce IV salt and challenge

Value	Purpose	Usually secret	Main requirement
Secret key	Controls encryption authentication or signing	Yes	Unpredictable to attacker and protected
Public key	Public encryption or verification input	No	Correct binding to intended party
Seed or generator state	Controls random generator	Often yes	Sufficient entropy and state protection
Nonce	Distinguishes a use or transaction	Usually no	Uniqueness in the required scope
CBC IV	Randomizes first block	No	Fresh unpredictability for encryption
CTR input prefix and counter	Identifies keystream blocks	No	No repeated complete input under one key
Password salt	Separates password hashing instances	No	Enough distinct values to prevent shared precomputation
Authentication challenge	Makes a response fresh	No	Freshness and unpredictability where required

Requirements vary by protocol. A value can need uniqueness without secrecy. A key can remain secret while a repeated nonce still destroys confidentiality.

6.6 Random collisions and the birthday effect

For k independent uniform draws from a space of N values, the probability of at least one repetition is approximately 1 - exp(-k*(k-1)/(2N)). At probability one half, k is about 1.177*sqrt(N).

For an n bit nonce, N = 2^n, so random repetitions become significant around 2^(n/2) draws, not 2^n. A correctly managed counter has no repeats until its space is exhausted, but resets, concurrent devices, or reuse with the same key can break that property. This is especially relevant to the IV question in CS-Answers.pdf.

6.7 One way and trapdoor functions

A **one way function** is easy to evaluate but hard for an efficient attacker to invert on the relevant input distribution. For a hash, inversion means finding an input giving a target digest, not necessarily recovering a unique original input.

A **trapdoor function** adds secret information that permits efficient inversion. Textbook RSA's public exponentiation is a teaching example: an owner who knows the suitable private exponent can invert it. Knowing prime factors permits that private exponent to be computed. Hashes have no intended secret inversion key.

Concrete one way example for a short answer: in a suitable large cyclic group, f(x) = g^x is easy to compute by repeated squaring, while recovering x is the discrete logarithm problem, assumed hard for the selected parameters. In the tiny teaching group modulo 7, 3^4 = 4 mod 7; enumeration recovers 4 easily, so this tiny example illustrates the operation rather than security. For a trapdoor example, give RSA f(x) = x^e mod n, inverted with d as f(x)^d mod n. State the hardness assumption and the trapdoor; do not use simple squaring on ordinary integers as a supposedly hard function.

Security is computational, not a claim that no possible attack exists. With finite key spaces and a way to test guesses, exhaustive search is possible in principle. The practical question is whether its cost exceeds the attacker's feasible resources and the useful lifetime of the secret.

6.7.1 Discrete logarithms and quadratic residuosity

Supporting definition for DLP; extension for QRP. September 2025 Part B Q8 asks you to state both problems, with only a quarter mark each. Understand the distinction before memorizing a concise answer.

Discrete logarithm problem: given a cyclic group, generator g, and group element y = g^x, find x modulo the order of g. In elliptic curve notation, given G and Y = xG, find x. This is different from computational DH, which asks for g^(ab) given g^a and g^b, or abG given aG and bG.

A nonzero invertible a is a **quadratic residue modulo n** if some x satisfies x^2 = a mod n. For example, 2 is a residue modulo 7 because 3^2 = 2 mod 7. The nonzero residues modulo 7 are 1, 2, and 4. Testing whether a root exists is a decision problem; actually finding a root is a search problem.

Quadratic residuosity problem, cryptographic form: given an odd composite modulus n with hidden factorization and an invertible a whose Jacobi symbol is +1, decide whether a is a square modulo n. The Jacobi symbol is an efficiently computable sign combining the prime factor symbols; a value +1 is necessary but not sufficient for being a square modulo a composite. You need not calculate this symbol for the paper's definition question.

The hard cryptographic setting is important. For an odd prime p, Euler's criterion efficiently tests whether a nonzero a is a residue: calculate a^((p-1)/2) mod p, which equals 1 for a square and -1 for a nonsquare. Known factors of an appropriate composite also permit efficient testing. Do not claim this prime test solves the unknown factorization version. These distinctions follow the Handbook of Applied Cryptography, Chapter 3.

Quarter mark versions: DLP asks for x from g and g^x. QRP asks whether a given a is a square modulo a supplied composite n, in the cryptographic setting with unknown factors. Section 18.28 gives a small worked illustration.

6.7.2 Calculating modular square roots

Official notes pages 9, 10, 19, and 20; extension beyond the named algorithm outlines. First distinguish asking whether a square root exists from calculating it. Modulo an odd prime, every nonzero square has exactly two roots: if r is one, p-r is the other. This follows from x^2-r^2 = (x-r)(x+r) and the absence of zero divisors in a field. Zero has just the root zero.

For p = 11, the nonzero squares are {1,3,4,5,9}; the nonsquares are {2,6,7,8,10}. Euler's criterion tests membership using a^5: the result is 1 for a square and 10, representing -1, for a nonsquare. It is a membership test, not automatically the root itself.

When **p = 3 mod 4** and a is a nonzero quadratic residue, a root is r = a^((p+1)/4) mod p. The exponent is an integer. Proof: r^2 = a^((p+1)/2) = a*a^((p-1)/2) = a, using Euler's criterion for a residue. The second root is p-r. If a is a nonsquare, the same calculation does not produce a root of a; check the condition first.

For p = 11 and a = 3, r = 3^3 = 27 = 5 mod 11, and the other root is 6. Verify both: 25 and 36 have remainder 3. When p = 1 mod 4, this particular exponent shortcut is unavailable. **Tonelli-Shanks** is the named general method in the notes: write p-1 = 2^s*q with q odd, find a nonsquare, and iteratively adjust a candidate using powers of that nonsquare until its square equals a. Learn its purpose and the distinction; the supplied notes do not derive its full algorithm.

For composite n with known suitable prime factors, solve in each prime field and use CRT. Without the factors, finding roots for suitable composite moduli is a separate cryptographic hardness setting. Do not apply the prime formula just because n happens to be odd. The prime shortcut and composite distinction are checked against Shoup's number theory text, Chapter 12.

6.8 Complexity vocabulary from the slides

P consists of decision problems solvable in deterministic polynomial time. **NP** consists of decision problems whose yes instances have polynomial size certificates verifiable in polynomial time. NP does not stand for not polynomial, and it does not imply proven hardness. P is contained in NP.

NP hard means every NP problem can be reduced to the problem in polynomial time. Such a problem need not itself be in NP or even be a decision problem. **NP complete** means both in NP and NP hard. SAT is a standard example. The general traveling salesman decision problem is NP complete; its optimization form is NP hard.

A reduction from A to B means an efficient solution to B would permit an efficient solution to A. To show a new problem B is NP hard, reduce known hard problems to B, not B to an easy problem.

Corrections: P not equal NP would not alone prove secure cryptography exists. Cryptography needs appropriate average case hardness, not only hard worst cases. Integer factorization is not known to be NP complete. A problem's input size means its bit length, so exponential growth must be discussed relative to that size. A fixed nine by nine Sudoku board illustrates verification intuitively; complexity claims formally concern families of growing inputs.

6.8.1 Deterministic and randomized algorithms

Official notes pages 1, 3, 4, and 15; supporting vocabulary. A deterministic algorithm makes the same choices on the same complete input. A randomized algorithm also uses random choices, so its path may vary. A seeded PRNG remains deterministic when its initial seed and state are fixed. Randomized is not the same word as nondeterministic in the definition of NP.

A **Monte Carlo** algorithm has a controlled time bound but may return an incorrect answer with a bounded probability. Independent repetition can reduce appropriate error probabilities. Some texts reserve Monte

Carlo for one sided error; check the convention, but retain the time versus error distinction. A **Las Vegas** algorithm returns a correct answer when it finishes, while its running time depends on random choices; efficiency is usually expressed through expected running time. Randomness alone proves neither speed nor correctness.

For a reduction A <= B, a solver for B helps solve A. Consequently, if A is hard under the relevant definition, B cannot be easy. The reverse conclusion does not follow: hard B does not by itself establish hard A. In crypto, specify the problems and type of attack rather than treating all hard problems as interchangeable. DLP solving gives DH solving; factoring gives RSA key recovery. Neither statement proves a converse.

6.8.2 Security parameters and negligible probability

Official notes page 15; supporting conceptual depth. The security parameter k indexes growing problem sizes. A **probabilistic polynomial time (PPT)** attacker uses random choices and a running time bounded by a polynomial in the relevant input length or security parameter. It is an idealized efficient attacker used in formal definitions.

A success probability is **negligible in k** if it eventually falls below 1/k^c for every positive constant c. Exponential decay such as 2^(-k) is negligible; 1/k or 1/k^100 is not, even though either can be small at some sizes. This is an asymptotic definition, not merely the adjective unlikely.

If one secret is uniformly chosen from 2^k candidates, q independent uniform guesses with replacement succeed with probability 1-(1-2^(-k))^q, at most q/2^k. For polynomial q this bound is negligible. With q distinct guesses the probability is exactly q/2^k for q <= 2^k. These are models of guessing a uniform secret, not proofs that factoring a k bit number requires brute forcing every k bit string. Cryptographic security must also rule out useful structural attacks.

6.9 Correctness and security are separate

The slides use **completeness** for honest behavior succeeding and **soundness** for cheating failing. In encryption, the usual terminology is correctness and confidentiality security. For signatures, use correctness and unforgeability. Formal soundness is particularly natural in proof and authentication settings.

A cipher that returns the input unchanged is perfectly correct to decrypt but provides no secrecy. A verifier that accepts every signature accepts honest signatures, but also every forgery. Mathematical correctness therefore cannot establish adversarial security.

Checkpoint: Why can a timestamp seeded random generator defeat AES security? **Answer:** an attacker may enumerate plausible seeds and derive the key, bypassing search through the full AES key space. Why does a KDF not create new entropy? **Answer:** deterministic processing cannot make the unknown password choice less guessable; it can make each guess more expensive.

7 Symmetric asymmetric and hybrid cryptography

Core. Sources: Preliminaries slides 33 to 40; the symmetric and asymmetric decks; C1 2023 question 1.

7.1 Symmetric cryptography

The communicating parties share a secret key. That secret controls encryption and decryption or MAC generation and checking. AES is a modern block cipher. DES is a historical block cipher. Symmetric methods are efficient for large volumes of data.

The difficulty is distributing and protecting the shared secret. If n users each need a distinct key for every pair, the number of pairwise keys is n(n-1)/2. For 100 users this is 4,950. This formula assumes a distinct key per unordered pair. A central service or another architecture can use a different arrangement.

A shared key also limits attribution. Either holder can generate the same valid MAC, so a third party cannot tell which holder produced it solely from that MAC.

7.2 Asymmetric cryptography

Asymmetric mechanisms use related public and private keys. For public key encryption, a sender uses the **recipient's public key**, and the recipient decrypts with the **recipient's private key**. For signatures, the signer uses the **signer's private key**, and the verifier uses the **signer's public key**.

DH is a public key technique for agreement on a shared secret; it is not itself message encryption or a signature. Not every asymmetric scheme supports all these operations. The claim that either key encrypts and the other decrypts is a shorthand for some textbook RSA operations, not a universal rule for public key cryptography.

Public keys can travel openly, but they must still be associated with the intended party. Substituting an attacker's public key is an identity problem. This module explains the problem and assumes a trustworthy authentication method when needed; PKI is excluded.

7.3 Compare the two families

Feature	Symmetric	Asymmetric
Secret material	Shared secret	Private key for each owner
Public material	Algorithm and allowed public parameters	Algorithm and public keys
Common role	Bulk encryption and MACs	Key establishment and signatures
Performance	Generally efficient for large data	Usually more costly for comparable bulk tasks
Key management challenge	Sharing secrets with intended parties	Authenticating public keys and protecting private keys
Publicly verifiable signatures	Shared key MAC does not provide them	Signature schemes provide them
Examples	AES DES HMAC	RSA DH and ECC schemes

Compare equivalent security, not equal numerical key lengths. A 256 bit ECC key and a 256 bit AES key have different structures and do not represent the same brute force task. Avoid memorizing an exact speed multiplier; hardware, implementation, and operation matter.

7.4 Hybrid design

Use an authenticated asymmetric mechanism to establish a session secret. Derive the required encryption and authentication keys from it. Use a symmetric authenticated encryption construction for application data. Rotate or discard session keys appropriately.

```
Authenticated key establishment
              |
       shared session secret
              |
       key derivation function
              |
  symmetric traffic protection keys
              |
    protected application records
```

This combines open network key establishment with efficient data protection. Modern TLS is a familiar example, but a full handshake is outside the scope. The word hybrid is also used for classical plus post quantum key establishment, a different combination explained in Chapter 17.

7.4.1 Hybrid key transport in the class diagram

Official notes page 37; core combination of symmetric and asymmetric methods. Key agreement such as DH is one way to establish a secret. Another is **key transport**: Alice generates a fresh secret K, protects K for Bob using his authentic public encryption key, and encrypts the long message using K.

```
ALICE                                        BOB
Fresh random K
C_key  = Encrypt(Bob_public, K) ----------> K = Decrypt(Bob_private, C_key)
C_data = SymmetricEncrypt(K, M) ----------> M = SymmetricDecrypt(K, C_data)
```

Only the short key is sent through the expensive public key operation. The private decryption key belongs to Bob; it is not sent or used as Alice's random seed. In a modern realization, use a specified randomized public key scheme such as RSA OAEP or a KEM, and symmetric authenticated encryption with correct nonce handling. The diagram abstracts encoding, nonce, and authentication tag, which must be transmitted or derived as the chosen scheme specifies. Confidential transport to Bob does not alone authenticate Alice as sender; use an appropriate additional authentication mechanism if that is required.

7.5 Block and stream ciphers

A **block cipher** maps a fixed size plaintext block to an equal size ciphertext block under a key. For each key, the mapping must be a permutation so decryption can reverse it. AES always operates on 128 bit blocks. A mode defines how to use it on longer messages.

A **stream cipher** generates a keystream and combines it with data, commonly by XOR. C = M XOR Z, and M = C XOR Z. The keystream must be unpredictable to an attacker and must not be reused across messages under the relevant key and nonce.

Padding is mode dependent: CBC generally needs complete blocks and padding, while CTR uses a block cipher as a stream construction and can handle a partial final block. A block cipher's classification does not mean every use requires padding.

Short comparison for September 2025 Part B Q11: block cipher modes offer flexibility for stored or bulk data, but some require padding and propagate ciphertext errors across blocks. Stream constructions can process data incrementally with low buffering and no block padding, but require careful nonce management and synchronization; keystream reuse is disastrous. Speed, parallelism, and error

propagation depend on the chosen construction, so neither family is universally faster or safer. Neither bare family automatically authenticates its ciphertext.

Synchronous stream construction: the keystream is independent of ciphertext, so both parties must maintain position. OFB and CTR illustrate the idea. **Self synchronizing or asynchronous construction:** the state depends on recent ciphertext, so synchronization can recover after a finite number of ciphertext units. CFB is the standard example.

LFSR and NLFSR terminology, official notes page 40. A linear feedback shift register stores bits, shifts them, and computes feedback as a linear combination, typically XOR of selected state bits. It can generate a long sequence cheaply, but a lone linear recurrence is predictable from enough output and is not a CSPRNG. A nonlinear feedback shift register uses nonlinear feedback; that label alone also does not prove cryptographic security. For the syllabus, connect these examples to the need for an unpredictable keystream rather than memorizing a circuit design.

7.6 The reused keystream problem

Suppose C1 = M1 XOR Z and C2 = M2 XOR Z use the same keystream. Then C1 XOR C2 = M1 XOR M2, because Z cancels. If one plaintext is known, Z can be recovered for that portion and the other plaintext recovered too.

For a toy byte, let M1 = 0xAA, Z = 0x3C, and C1 = 0x96. Anyone knowing M1 and C1 can compute Z = 0xAA XOR 0x96 = 0x3C. This reveals that keystream byte, not necessarily the generator's entire secret key. The distinction matters in the 2025 RC4 question.

Checkpoint: Alice wants Bob to read a secret message and verify she sent it. Which keys and mechanisms? **Answer:** use a signing mechanism with Alice's private key and verify with her public key, plus encryption addressed to Bob or a mutually authenticated shared session key. A signature alone does not hide the message.

8 DES AES and cipher design

Core for algorithm outlines; supporting for components and diagrams. Sources: Symmetric Cryptography slides 21 to 35; Quiz_20 question 1; Mid Sem_18 printed 2020 question 5; Quiz1-Diagrams pages 3 to 6.

8.1 Confusion diffusion and avalanche

Confusion makes the relation between a key and ciphertext complex, usually through nonlinear substitutions. **Diffusion** spreads the effect of input data across many output bits, using permutations and mixing layers. A straight bit permutation only moves positions; repeated substitution and mixing make each input influence many outputs.

The **avalanche effect** is the statistical expectation that changing one input or key bit changes roughly half the output bits. It is measured over many inputs and bit changes, not guaranteed as an exact half for every pair. Good avalanche behavior is necessary evidence of useful mixing, but is not by itself a security proof.

8.2 Primitive components

An **S box** substitutes a bit pattern using a table or equivalent computation. DES S boxes map six bits to four. The AES S box maps eight bits to eight and is invertible. AES builds it using a field inverse and an affine transformation, with a specified treatment of zero.

A **P box** permutes positions. A straight permutation preserves width and is invertible. Expansion duplicates selected bits; compression selects fewer bits. The latter two are not bijective on their own. These are components, not necessarily full cipher transformations.

XOR mixes a key or another state into the data. **Rotations** move bit positions reversibly. A **key schedule** derives round keys from the main key. Round keys are related through that schedule, not independent newly sampled keys.

8.3 Feistel structure

Split a block into two halves L and R. One round is:

```
L_i = R_(i-1)
R_i = L_(i-1) XOR F(R_(i-1), K_i)
```

To recover the previous halves, R_(i-1) = L_i, and L_(i-1) = R_i XOR F(L_i,K_i). Thus the round is reversible even if F itself is not invertible. Encryption and decryption can use the same structural pattern with round keys reversed and the appropriate end conventions.

Toy calculation: L0 = 1010, R0 = 0110, and F(R0,K1) = 1100. Then L1 = 0110 and R1 = 1010 XOR 1100 = 0110. To invert, recover R0 = 0110 and L0 = 0110 XOR 1100 = 1010.

8.4 DES overview and diagram

DES uses **64 bit blocks**, a **56 bit effective key**, and **16 Feistel rounds**. A supplied DES key is commonly represented as 64 bits with eight parity bits; those parity bits do not add cryptographic search space.

```
64 bit plaintext
        |
Initial permutation IP
        |
L0 32 bits                R0 32 bits
        |                       |
        +---- 16 Feistel rounds --+
        |
Combine as R16 followed by L16
        |
Inverse initial permutation IP^(-1)
        |
64 bit ciphertext
```

The initial and final permutations rearrange bits and reverse one another. They are not the source of the cipher's secrecy. The 16 keyed rounds provide the main mixing.

8.5 DES round function

```
R input 32 bits
        |
Expansion E to 48 bits
        |
XOR round key K_i 48 bits
        |
Eight S boxes in parallel
Each consumes 6 bits and outputs 4 bits
        |
32 output bits
        |
Straight permutation P
        |
F output 32 bits
```

Expansion duplicates selected positions, matching the subkey width and spreading influence. XOR introduces key dependence. The S boxes introduce nonlinear substitution and return the width to 32 bits. The P permutation routes bits into different S box inputs in later rounds, aiding diffusion.

For a DES S box input b1 b2 b3 b4 b5 b6, choose the row from the outside bits b1 b6 and the column from the middle four b2 b3 b4 b5. For input 101011, the row bits are 11, row 3, and the column bits are 0101, column 5. The actual four bit result depends on which of the eight specified S box tables is used.

8.6 DES key schedule

Start from the 64 bit supplied key. PC 1 drops the eight parity bits and permutes the remaining 56. Split into C0 and D0 of 28 bits each. For each round, circularly shift each half left, concatenate the halves, and apply PC 2 to select 48 bits for the round key.

The left shifts are one bit in rounds 1, 2, 9, and 16, and two bits in the other rounds. If asked for the diagram, label 64 to 56, the two 28 bit halves, the circular shifts, and 56 to 48 for each subkey. These widths earn marks and prevent confusing the main key with a round key.

8.7 Why DES was replaced

A 56 bit key provides 2^56 candidates, too small for modern security expectations. Its 64 bit blocks also impose limits when large amounts of data are processed under a key. Increasing computing capability made brute force practical. The important exam explanation is the limited effective key size, not an exact cracking time for unspecified hardware.

Triple DES historically extended DES use by applying it multiple times. It is legacy material here. AES supplies a larger block and stronger key options. Do not propose DES or RC4 as a new secure design merely because they are taught in class.

8.8 AES overview

AES uses a **128 bit block in every variant**, with keys of **128, 192, or 256 bits** and **10, 12, or 14 rounds**, respectively. It is a substitution permutation network rather than a Feistel cipher. The complete state, represented as a four by four array of bytes, is transformed.

The NIST AES standard specifies the three variants. The AES numbers below and the teaching sequence follow your supplied slides.

```
16 plaintext bytes arranged as the state
                    |
Initial AddRoundKey using round key 0
                    |
Rounds 1 through N_r minus 1
SubBytes then ShiftRows then MixColumns then AddRoundKey
                    |
Final round N_r
SubBytes then ShiftRows then AddRoundKey
                    |
16 ciphertext bytes
```

The final round omits MixColumns. There is an initial key addition before the first full round. AES 128 therefore needs **11 round keys**, including the initial one, not 10.

8.9 The four AES transformations

SubBytes: replace every byte using the invertible S box. This supplies nonlinearity and confusion. The GF(2^8) inverse is part of the S box construction, not ordinary division modulo 256.

ShiftRows: rotate rows left by 0, 1, 2, and 3 byte positions. This moves bytes between columns, allowing later column mixing to spread their effect throughout the state.

MixColumns: transform each four byte column using a fixed invertible linear operation in GF(2^8). Each output byte depends on multiple input bytes. It is a diffusion layer.

AddRoundKey: XOR the state with the round key. It is the only one of the four round transformations that directly injects the key. XOR makes it reversible by applying the same round key again.

Decryption applies corresponding inverse transformations with an appropriate reversed key sequence. AES round operations need invertibility; unlike Feistel F, its complete round cannot simply discard information.

8.10 AES key expansion outline

The expansion generates enough four byte words for every round key. In AES 128, start with four words from the original key. For each new word, XOR a previous word with a word four positions earlier. At each group boundary, transform the previous word with **RotWord**, **SubWord**, and a round constant **Rcon** before XORing. AES 192 and AES 256 use adjusted schedules.

For the syllabus outline, know why expansion is needed, which key sizes produce which round counts, and the roles of rotation, byte substitution, and round constants. There is no need to memorize all expanded keys or compute a full AES ciphertext by hand.

8.11 Compare DES and AES

Attribute	DES	AES
Block size	64 bits	128 bits
Effective key sizes	56 bits	128 192 or 256 bits
Rounds	16	10 12 or 14
Structure	Feistel	Substitution permutation network
Nonlinear component	Eight 6 to 4 S boxes in F	8 to 8 S box in SubBytes
Decryption idea	Same structure with reversed subkeys	Inverse transformations with suitable key order
Exam security explanation	Key space too small	Strong modern primitive when used correctly

AES security still depends on the mode, keys, nonce handling, and implementation. A strong cipher in ECB can leak patterns; a leaked AES key reveals the protected data; side channels can target implementations.

8.12 How to answer the DES structure question

Begin with the block, effective key, and round counts. Draw the top level structure. Write the Feistel equations. Draw F with every width. Explain the key schedule and reversed subkey decryption. Finish with the weakness of the 56 bit key. This is a fuller answer than a diagram with unlabeled boxes.

Checkpoint: Why can a DES S box shrink six bits to four without making DES decryption impossible? **Answer:** it lies inside a Feistel function; the round preserves information across both halves and does not require F to be invertible. Why does AES 256 still encrypt only 128 bits per primitive call? **Answer:** AES key size and block size are separate parameters.

9 Block cipher modes and their failure patterns

Supporting from slides and past papers. Sources: Symmetric Cryptography slides 38 to 43; Mid-Sem_16 printed 2018 question 2c. The written syllabus asks for algorithm outlines; these modes explain how those algorithms protect real messages.

9.1 Why a mode is needed

AES defines a permutation of one 128 bit block for a given key. A mode specifies how to process a sequence of blocks, use an IV or nonce, and handle a partial final block. The mode changes pattern leakage, parallel processing, and the consequences of altered ciphertext.

Use P_i for plaintext block i, C_i for ciphertext, E_K for the encryption permutation, and D_K for its inverse. The following five traditional modes primarily address confidentiality. They do not themselves give message authenticity.

9.2 ECB

Electronic Codebook: C_i = E_K(P_i), and P_i = D_K(C_i). Blocks are independent. Encryption and decryption can run in parallel. Identical plaintext blocks under the same key produce identical ciphertext blocks, revealing repeated structure.

An attacker can also rearrange or replace whole ciphertext blocks without an integrity check. ECB illustrates why encrypting each block separately is insufficient for ordinary structured messages. Do not recommend it for general data protection.

9.3 CBC

Cipher Block Chaining: let C_0 = IV. Then:

```
C_i = E_K(P_i XOR C_(i-1))
P_i = D_K(C_i) XOR C_(i-1)
```

The previous ciphertext enters the next encryption, making the output depend on earlier blocks. The first block uses a fresh unpredictable IV. The IV is transmitted openly but must be handled with the needed integrity protection.

Encryption is sequential because C_i is needed for the next block. Decryption can process blocks in parallel because each C_i and preceding ciphertext are already available. Padding is normally needed to fill the final block.

Bit error: if one bit of C_i changes, decryption of P_i is unpredictably corrupted, and the same bit position in P_(i+1) flips. Later blocks recover, assuming block boundaries remain aligned. Changing the IV predictably flips bits in the first plaintext block. This is a direct reason authentication is needed.

9.4 CFB

For **full block Cipher Feedback**, let C_0 = IV:

```
C_i = P_i XOR E_K(C_(i-1))
P_i = C_i XOR E_K(C_(i-1))
```

Both directions use E_K to produce the XOR mask. Ciphertext feeds back. Encryption is sequential; full block decryption can parallelize when ciphertext is present. CFB can also use smaller segments, in which case feedback and error duration differ.

For a one bit error in full block C_i, the corresponding bit in P_i flips; P_(i+1) is unpredictably corrupted; later blocks recover. This is more precise than saying both blocks are entirely corrupted. The construction is self synchronizing after enough correctly received ciphertext segments.

9.5 OFB

Output Feedback: O_0 = IV, O_i = E_K(O_(i-1)), and C_i = P_i XOR O_i. Decryption uses the same XOR. The feedback is the internal output, not the ciphertext.

The output chain is sequential but can be generated before plaintext arrives. A changed ciphertext bit changes only the corresponding plaintext bit. A lost bit or byte can disrupt synchronization, so limited error propagation is not the same as automatic recovery from inserted or removed data.

Never repeat the generated keystream under a key. Reusing an IV with the same key repeats the sequence and exposes relationships between plaintexts.

9.6 CTR

Counter mode: construct a distinct block T_i from a nonce or prefix and a counter. Compute Z_i = E_K(T_i), then C_i = P_i XOR Z_i. Encrypting the same distinct counter blocks reconstructs the mask for decryption.

Encryption and decryption can parallelize, and a block can be accessed without decrypting earlier blocks. Padding is unnecessary for a partial final XOR segment. A changed ciphertext bit changes the corresponding plaintext bit.

Critical rule: never repeat an entire T_i input under the same key, even across different messages. A reset counter with a reused nonce can repeat keystream. The nonce is public; its secrecy does not solve input reuse.

9.7 Comparison and authenticated encryption

Mode	Encryption parallel	Decryption parallel	Typical padding	One ciphertext bit error
ECB	Yes	Yes	Needed for ordinary complete block use	Unpredictable corruption in that plaintext block
CBC	No	Yes	Yes	Current block corrupted next block same bit flips
Full block CFB	No	Yes	No block padding for appropriate segments	Current bit flips next block corrupted
OFB	Output generation sequential	Output generation sequential	No	Only corresponding plaintext bit flips
CTR	Yes	Yes	No	Only corresponding plaintext bit flips

The NIST modes specification defines these confidentiality modes. Your exam needs equations, feedback source, IV or nonce rule, parallelism, and error pattern, rather than implementation code.

Authenticated encryption with associated data or AEAD protects confidentiality and detects modification, including selected unencrypted metadata. AES GCM is an example combining a counter

style encryption construction with authentication. Correct nonce management remains essential. Verify authentication before relying on decrypted content.

Checkpoint: Why do OFB and CTR's limited error propagation not make them tamper resistant? **Answer:** an attacker can deliberately flip ciphertext bits to change plaintext bits predictably; an authentication tag detects that alteration.

9.8 Two encryption diagrams to reproduce in an exam

September 2025 Part B Q15 asks for any two modes, encryption only, with block diagrams. The equations and mode comparisons above remain the lesson. These diagrams provide a compact drawing answer. E_K is block encryption under key K; XOR is bitwise exclusive OR. Label the plaintext, ciphertext, and the IV or counter input.

CBC encryption: the preceding ciphertext enters the XOR before block encryption. For the first block substitute the IV for C_0.

```
         C_(i-1)  (IV when i=1)
              |
              v
P_i ------> [XOR] ------> [E_K] ------> C_i
                                        |
                            feeds the next block's XOR

C_i = E_K(P_i XOR C_(i-1)),  C_0 = IV
```

Explain that chaining hides repeated block patterns, encryption is sequential, and the first IV must meet CBC's unpredictability requirement. CBC alone supplies no authentication.

CTR encryption: encrypt a unique nonce and counter block, then XOR the output with plaintext. Ciphertext does not feed the next block.

```
T_i = nonce || counter_i ---> [E_K] ---> Z_i
                                         |
                                         v
P_i --------------------------------> [XOR] ---> C_i

C_i = P_i XOR E_K(T_i)
```

Explain that independent counter inputs permit parallel encryption and a partial final XOR segment needs no block padding. Never repeat T_i under the same key. For a two mark question, two labeled diagrams and their defining equations give a focused answer; add one distinguishing property if space allows.

10 Hash functions MD5 SHA 256 and the birthday bound

Core with supporting construction details. Sources: Symmetric Cryptography slides 3 to 14; C1 2023 question 2; older midsemester and endsemester collision questions.

10.1 A fingerprint rather than encryption

A cryptographic hash takes a variable length input and produces a fixed length digest. A SHA 256 digest is 256 bits, or 32 bytes, commonly written as 64 hexadecimal characters. The same exact input produces the same digest. Changing a byte, whitespace, or encoding may change the digest.

Hashes are many to one because there are far more possible inputs than outputs. Collisions must exist. Security means an efficient attacker cannot find useful ones within feasible resources, not that every input has a unique digest.

A hash has no decryption operation. An input cannot in general be uniquely recovered from a digest. An attacker can still guess likely inputs and hash each guess. This explains why fast SHA 256 alone is unsuitable for password storage.

10.2 The three security properties

Preimage resistance: given a target digest h, finding any x with H(x) = h is infeasible. This is a target output search. For an ideal n bit output, generic search costs approximately 2^n evaluations.

Second preimage resistance: given a specific message x, finding a different y with H(y) = H(x) is infeasible. The first message is fixed. An example is substituting a malicious file for an existing file while preserving its trusted hash.

Collision resistance: finding any pair of distinct x and y with H(x) = H(y) is infeasible. The attacker chooses both inputs. An example is preparing two different contracts with the same hash, having one signed, and substituting the other.

Exam wording: soundness requirements of a secure hash. The September 2025 paper uses this phrase. State preimage resistance, second preimage resistance, and collision resistance, with the three different attacker tasks above. Determinism, efficient evaluation, variable length input, and fixed length output describe the function; avalanche behavior is desirable, but does not replace those adversarial security requirements.

Property	Attacker starts with	Attacker must find	Ideal generic effort
Preimage	Target digest h	Any x matching h	About 2^n
Second preimage	Specific x	Different y matching H(x)	About 2^n for usual target sizes
Collision	Freedom to choose inputs	Any distinct matching pair	About 2^(n/2)

Do not assert a universal implication chain across all three. Collision resistance strongly relates to second preimage resistance under usual models, because a successful substitution supplies a collision. Preimage resistance is a separate requirement. Formal details depend on the function family and the security experiment.

10.3 Why collisions need fewer attempts

With k sampled digests, there are k(k-1)/2 pairs to compare. Each pair has probability 1/2^n of matching in an ideal model. The number of pairs grows quadratically even though the number of samples grows linearly.

Thus a significant collision chance appears around sqrt(2^n) = 2^(n/2) samples. For a 256 bit ideal hash, generic collision effort is about 2^128 and target preimage effort about 2^256. A broken algorithm may permit much faster attacks than these ideal bounds.

For two independent sets of a and b random messages, expected cross set matches are ab/2^n, and the approximate probability of at least one cross collision is 1 - exp(-ab/2^n). This differs from collisions within one set, which use k(k-1)/2 pairs. Chapter 18 derives the two past paper formulas.

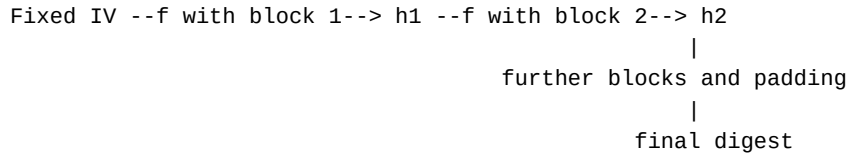
10.4 How a hash can support integrity

Download a file and compare H(file) with a reference obtained through a trusted source. A difference establishes that the file differs from the referenced content. A match offers evidence tied to the hash's security and the trustworthiness of that reference.

If an attacker can replace both the file and the displayed hash, the attacker simply calculates a new hash. A hash sent beside an unauthenticated message therefore does not authenticate the sender. Use a MAC, signed digest, or authenticated distribution channel when an active adversary is present.

10.5 Merkle Damgard construction

MD5 and SHA 256 process fixed size message blocks through a compression function and a running chaining state. The message is padded, including its length, before processing.



The compression function receives the prior state and the next block, then produces a new state. Here IV means a fixed specified initial hash value, unlike the per message IV of CBC. Length encoding and padding are part of the construction, not optional decoration.

Padding for MD5 and SHA 256: append a 1 bit, enough zeroes to reach 448 modulo 512, then a 64 bit representation of the original bit length. The result is a multiple of 512 bits. Their length encoding byte orders differ. For a 24 bit input such as a three byte ASCII message, add 1 plus 423 zero bits plus 64 length bits, giving one 512 bit block.

This construction can expose **length extension**: from a digest and known message length, one may calculate a digest for a specifically padded message with an appended suffix. It does not invert the hash, but makes naive H(secret || message) unsuitable as a general MAC.

10.6 MD5 outline

MD5 outputs **128 bits** and takes **512 bit message blocks**. Its state has four 32 bit words. Each block undergoes **64 operations organized in four groups of 16**. Operations mix message words, state, constants, Boolean functions, additions modulo 2^32, and left rotations. The resulting state is added to the prior chaining state before the next block.

MD5's collision resistance is broken. Do not treat its nominal 128 bit digest as providing the ideal 64 bit collision search security in adversarial use. Learn it for historical explanation and the syllabus outline, rather than proposing it for new signatures or integrity designs.

10.7 SHA 256 outline

SHA 256 outputs **256 bits** and processes **512 bit blocks**. Its state has eight 32 bit words. A message schedule expands sixteen input words into sixty four words. Each block uses **64 rounds** of bitwise mixing, rotations, shifts, constants, schedule words, and additions modulo 2^32. The result updates the eight word chaining state.

For the required outline, draw input padding, 512 bit block processing, eight word state, 64 round compression, and 256 bit output. The NIST Secure Hash Standard is the official specification.

Supporting round vocabulary: Ch chooses bits from two words based on a selector word; Maj computes the bitwise majority of three words. Uppercase Sigma functions combine rotations of working words; lowercase sigma functions combine rotations and shifts for the message schedule. You do not need to memorize all constants for the syllabus's example hash outline.

10.8 Compare MD5 and SHA 256

Feature	MD5	SHA 256
Digest	128 bits	256 bits
Message block	512 bits	512 bits
Chaining words	Four 32 bit words	Eight 32 bit words
Compression	Four groups of 16 operations	64 rounds
Broad structure	Iterated compression with length padding	Iterated compression with length padding
Collision security status	Broken	No practical full function collision attack established in these materials

The archive's diagram sheet page 1 shows **SHA 512**, not SHA 256. SHA 512 uses 1024 bit blocks and a 512 bit digest. Do not copy those widths into a SHA 256 answer.

10.9 Password hashing

Store a per account salt and a verifier from a deliberately expensive password hashing mechanism. On login, compute the same function from the offered password and compare. The salt can be public. It prevents identical passwords from automatically producing identical stored verifiers across accounts and makes precomputed tables less reusable.

A salt does not make a weak individual password unguessable. An attacker with a stolen database knows the salt and can still guess that user's password. Cost parameters make each guess more expensive; memory hard designs can limit highly parallel attackers. Password reuse still permits credential stuffing at another service.

Checkpoint: An attacker chooses two contracts that collide, then asks you to sign one. Which property fails? **Answer:** collision resistance. An attacker starts with your existing signed contract and searches for another matching one. **Answer:** second preimage resistance.

11 MACs digital signatures and non repudiation

Core for integrity signatures and non repudiation; supporting for HMAC and CMAC. Sources: Symmetric Cryptography slides 14 to 17; Asymmetric Cryptography slides 21 to 25; Quiz_20 questions 3 and 4; C1 2023 questions 4 and 7.

11.1 A message authentication code

A **Message Authentication Code or MAC** calculates a tag T = MAC_K(M) using a shared secret key. The receiver recalculates or verifies the tag for the received message. An outsider without the key should be unable to make an altered message pass verification.

It provides integrity and origin assurance relative to the set of people holding the key. It does not encrypt M. It also does not establish which particular shared key holder generated a tag. Therefore it does not provide the same publicly verifiable evidence as a digital signature.

MAC means Message Authentication Code here. In Chapter 15, MAC means Mandatory Access Control. Always expand the abbreviation once when context could confuse the reader.

11.2 HMAC

HMAC builds a keyed tag from a hash using a nested inner and outer construction:

```
HMAC(K,M) = H((K0 XOR opad) || H((K0 XOR ipad) || M))
```

The symbol || means concatenation. K0 is the key normalized to the hash's block length: a long key is hashed, then the result is padded with zero bytes as needed. ipad repeats byte 0x36 and opad repeats byte 0x5C. For SHA 256, the hash block is 64 bytes, while the digest is 32 bytes.

The inner hash binds the message to a key dependent prefix. The outer keyed hash protects the inner result. This structure is designed to avoid the length extension weakness of simple secret prefix hashing. HMAC SHA 256 is a typical instantiation. Its definition is in RFC 2104.

Exam working: sender and receiver share K; sender computes the tag and sends M with T; receiver computes the expected tag over the exact received M; accept only if verification succeeds. Include identity, purpose, and transaction fields in the authenticated data if those are part of what must be protected.

11.3 CMAC

CMAC builds a MAC from a block cipher, commonly AES. It chains message blocks through encryption and uses derived subkeys to treat a complete final block differently from a padded incomplete block. This final block treatment prevents weaknesses in applying raw CBC MAC indiscriminately to variable length messages.

At outline level, distinguish **HMAC from a hash** and **CMAC from a block cipher**. Do not confuse the archive's nested MAC diagram with a complete CMAC algorithm diagram. Hardware support and the rest of a system's design may influence which is efficient.

11.4 Digital signatures

A signature scheme has **key generation, signing, and verification**. The signer uses a private key. Anyone with the corresponding authentic public key can verify. Modern signatures usually operate on an encoded representation involving a cryptographic digest and defined formatting.

The receiver verifies the signature against the exact received message. Changing the message or signature should cause verification failure. Authenticity assumes the verifying key belongs to the intended signer and that the signing key has remained controlled.

```
Signer: message --hash and encode--> representative
                                        |
                                private key signing
                                        |
                                   signature

Receiver: message and signature and authentic public key
                                        |
                                   verification
                                        |
                                 accept or reject
```

11.5 Signature services and limits

Integrity: unauthorized changes cause verification failure. **Origin authentication:** successful verification ties the signature to possession of the corresponding private key. **Non repudiation support:** a third party can verify evidence without holding the signing key.

This is not an unconditional proof of a person's intent. Compromised keys, shared signing credentials, malicious software, or an incorrectly attributed public key weaken the evidence. An ordinary signature does not automatically prove receipt, when someone read the message, or that its contents are true. A recipient must issue separate evidence if proof of receipt is required.

A valid signed message can be replayed. Include a transaction identifier, recipient, context, and freshness constraints when acceptance should occur only once. A signature verifies bytes; the application decides whether those bytes authorize the current action.

11.6 Compare hash MAC and signature

Question	Plain hash	MAC	Digital signature
Key involved	None	Shared secret	Private and public pair
Who can produce matching value	Anyone	Any shared key holder	Intended private key holder
Who can check	Anyone with trusted reference	Shared key holders	Anyone with authentic public key
Secrecy of message	No	No	No
Active modification protection	Needs trusted reference	Yes under MAC assumptions	Yes under signature assumptions
Third party attribution evidence	No	Limited because verifier can also generate	Supports it subject to key and identity assurance

Four algorithm classes versus four requirements, as requested in September 2025 Part B Q1: use the following compact answer. The qualifiers matter; a plain hash and encryption alone are not substitutes for authenticated integrity.

Algorithm class	Confidentiality	Origin authenticity	Integrity	Non repudiation
Encryption alone	Yes under a secure scheme	Not generally	Not generally	No
Plain hash alone	No	No	Detects changes only with a trusted reference	No
Message authentication code	No	Yes relative to shared key holders	Yes under MAC security	No
Digital signature	No	Yes with an authentic public key	Yes under signature security	Supports publicly checkable evidence

Authenticated encryption combines services; classify it as a combined construction rather than treating every encryption algorithm as automatically authenticating its ciphertext.

11.7 Checkpoint

Question: Why can't Bob prove that Alice sent a particular MAC tagged message to an outside judge?
Answer: Bob also knows the shared key and could have generated that tag. Public verification of a signature does not give Bob the private signing ability.

11.8 Brief digital certificate concept without PKI

Supporting scope clarification. Your syllabus separately names a digital certificate in the same line as PKI. This short overview preserves that concept; the infrastructure of authorities, issuance, trust chains, and revocation remains excluded. Skip this subsection if your teacher removed the whole combined line.

A **digital certificate** is a signed data structure associating an identity with a public key. A digital signature on a document proves a relationship to a key, but you also need a trustworthy association between that key and the intended signer. A certificate can supply signed evidence of that association under the relying party's trust assumptions.

The basic fields to recognize are the **subject identity**, **subject public key information**, **issuer identity**, **validity interval**, and **issuer's signature**. An X.509 certificate also includes a version, serial number, signature algorithm identifier, and optional extensions such as usage constraints. The outer structure contains the data to be signed, an algorithm identifier, and the signature value.

At this conceptual level, the verifier uses an already trusted issuer public key to check the signature on the certificate's signed content, and checks that the identity, time interval, and permitted use fit the intended application. Successful arithmetic verification by itself does not establish trust in an unknown issuer. It also does not prove that a website's content is honest or that its software has no vulnerabilities.

The distinction to remember is **message signature versus certificate signature**: the first authenticates a message under the signer's key; the second signs an identity and key association. Neither operation encrypts that message or key information. The machinery for establishing issuer trust at scale is PKI and is deliberately not taught here.

12 RSA from key generation to correctness

Core and highest calculation priority. Sources: Asymmetric Cryptography slides 14 to 25; C1 2023 questions 3 and 4; Quiz_20 questions 4 and 7; Mid Sem_19 question 1.

12.1 The idea

RSA uses modular exponentiation with a public exponent and a corresponding private exponent. Multiplying suitable large primes is efficient. Recovering their factors from the product is believed computationally difficult at suitable sizes. Knowing the factors lets the owner calculate the totient and the private exponent efficiently.

The numbers in hand calculations are intentionally tiny and insecure. They teach the mechanism, not an acceptable deployed key size.

12.2 Key generation

1. Choose distinct primes p and q.
2. Compute n = pq.
3. Compute phi(n) = (p-1)(q-1).
4. Choose e such that 1 < e < phi(n) and gcd(e,phi(n)) = 1.
5. Find d = e^(-1) mod phi(n), so ed = 1 mod phi(n).
6. Publish (n,e). Protect d and the private factor information.

More precise implementations may use the Carmichael value lambda(n), the least common multiple of p-1 and q-1, and additional CRT information. The totient method is appropriate for these exam examples.

The official notes choose d first (pages 32 and 33). After finding phi(n), choose an invertible d and calculate e = d^(-1) mod phi(n). This gives the same requirement ed = 1 mod phi(n) as choosing e first. In either route, **(n,e) is public and d is private**; the role is determined by which exponent is published, not by which was calculated first. The chosen exponent must be coprime to the totient; it does not have to be prime. Arbitrarily choosing a small private exponent is unsafe for real RSA. Section 18.37 reproduces the tiny class example solely for arithmetic.

12.3 Encryption and decryption

Represent a textbook message as an integer m with 0 <= m < n. Encrypt c = m^e mod n. Decrypt m = c^d mod n. Anyone with the public key can encrypt to the owner; knowing the public key does not grant the private exponent.

Encoding an arbitrary document into a suitable message is a separate task. RSA does not directly encrypt a multi gigabyte file by one exponentiation. Real systems use defined encoding and usually hybrid encryption.

12.4 Complete small example

Let p = 5, q = 11. Then n = 55 and phi(n) = 40. Choose e = 3; gcd(3,40) = 1. Since 3 times 27 = 81 = 1 mod 40, choose d = 27. Public key is (55,3), private exponent is 27.

Encrypt m = 7: c = 7^3 mod 55 = 343 mod 55 = **13**. To decrypt:

```
13^2 mod 55 = 4
13^4 mod 55 = 16
13^8 mod 55 = 36
13^16 mod 55 = 31
27 = 16 + 8 + 2 + 1
13^27 mod 55 = 31*36*4*13 mod 55 = 7
```

Reduce after every product to avoid large numbers. A quick independent check is that reencrypting recovered 7 gives 13.

12.5 Classic slide example

For p = 61 and q = 53, n = 3233 and phi(n) = 3120. Choose e = 17. Extended Euclid gives d = 2753, and 17 times 2753 = 46801 = 1 mod 3120. The message 65 encrypts to 2790 and decrypts back to 65.

You should understand each step rather than memorize this tuple. Chapter 18 works the actual smaller signature and inverse questions from the archive.

12.6 Correctness for coprime messages

Since ed = 1 + k*phi(n):

```
c^d = (m^e)^d = m^(ed)
    = m^(1 + k*phi(n))
    = m * (m^phi(n))^k
    = m mod n
```

The last step uses Euler's theorem and therefore requires gcd(m,n) = 1. This proves correctness for invertible messages.

12.7 Correctness for all message representatives

RSA also works for valid m divisible by p or q. The slide proof needs a small extension rather than pretending Euler's coprimality condition is absent.

Work modulo p. If m = 0 mod p, then m^(ed) = 0 = m mod p. Otherwise Fermat's theorem gives m^(p-1) = 1 mod p. Since ed = 1 mod p-1, m^(ed) = m mod p. Repeat the same argument modulo q. The two integers are equal modulo both distinct primes; the Chinese Remainder Theorem implies they are equal modulo pq.

This covers every m from 0 to n minus 1. For a short exam answer, give the Euler proof and explicitly state its condition; for a full correctness proof, add this prime by prime argument.

12.8 Correctness does not prove security

Factoring n reveals p and q, which permits calculating phi(n) and d. Therefore an efficient factorization algorithm breaks this method of protecting the private key. The converse is more subtle: it is not known that every possible RSA inversion attack must factor n.

The **RSA inversion assumption** concerns the difficulty of computing an e th root modulo n for appropriately generated keys and target values. Hard factorization alone is not a complete proof of semantic security for a deployed RSA encryption scheme. Padding, threat model, and implementation matter.

Do not write that Euler's theorem proves RSA soundness. It proves the relevant mathematical correctness. The C1 paper wording conflates these ideas; Chapter 22 explains how to answer accurately.

12.9 Why raw RSA is insecure

Determinism: the same m produces the same c. If an attacker is choosing between two known messages, the attacker encrypts each publicly and compares with the observed ciphertext. No factoring is needed.

Multiplicative malleability: from c = m^e mod n, form c' = c*r^e mod n. Its decryption is mr mod n. If a decryption oracle returns that value for c', an invertible r lets the attacker recover m. Chapter 18 applies this to the 2019 chosen ciphertext question.

Small and structured messages: values like 0 and 1 remain easily recognizable. This is another reason a mathematically correct primitive is not a complete secure encryption scheme.

The RSA specification in RFC 8017 defines RSA OAEP encryption and RSA PSS signatures, which add the required encoding structure. These are distinct schemes; the raw equations here are for learning and small exam calculations.

12.10 RSA signatures

For a toy textbook signature representative h, sign s = h^d mod n and verify that s^e mod n = h. In a real signature scheme, h represents a defined hash and encoding, not a direct instruction to raise the arbitrary document to a power.

Using the small key (n = 55, e = 3, d = 27) and representative h = 7, the signature is **28**, since 7^27 mod 55 = 28. Verify 28^3 mod 55 = 7. Encryption and signing use the private and public exponents for different purposes.

The correctness proof is the same exponent argument as encryption. **Unforgeability** is a separate claim. Raw RSA signatures have algebraic weaknesses and lack secure message formatting. A signature should not be described simply as encryption with the private key; its operation and security goal differ from encryption.

Hash then sign in the official notation (pages 34 and 35). The classroom model writes h = H(m), s = h^d mod n, and accepts when s^e mod n equals H(m). Treat h as an integer representative in the RSA domain. Correctness follows because (h^d)^e = h^(de) = h mod n, using the prime wise argument if h is not coprime to n. A complete scheme first applies its specified hash and encoding; a digest of arbitrary length is not automatically a valid representative below n.

The notes call honest acceptance completeness and resistance to forgery soundness. For a signature, the more precise security property is **unforgeability**, normally allowing the attacker to obtain signatures on other messages. Correctness and a collision resistant hash alone do not prove that property.

Why the phrase a forger must know d is too strong: for an entirely raw RSA signature on integer messages, anyone can choose s, compute h = s^e mod n, and produce a valid pair (h,s) without d. This does not directly give a signature on an arbitrarily selected document, but it already disproves that unrestricted raw claim. Hashing and secure encoding constrain which representatives correspond to documents; the complete construction still needs its own security argument. Learn the classroom equations, then state that deployed RSA signatures use a scheme such as RSA PSS.

12.11 Key ownership drill

- Confidential message to Bob: **Bob's public key** for encryption, **Bob's private key** for decryption.
- Signature by Alice: **Alice's private key** for signing, **Alice's public key** for verification.

- Signed and secret communication: combine the two goals through a defined secure protocol, rather than assuming either operation provides both.

Checkpoint: For p = 3, q = 5, e = 7, compute n, phi(n), and d. **Answer:** n = 15, phi = 8, d = 7. Check 7 times 7 = 49 = 1 mod 8. For representative m = 2, the textbook signature is 8 and verification gives 2.

13 Diffie Hellman key agreement

Core and high priority. Sources: Asymmetric Cryptography slides 26 to 31; C3 2023 question 4; Mid Sem_18 printed 2020 question 7; End-Sem_18 question 1a.

13.1 The problem it solves

Alice and Bob want a symmetric session key without first sending a secret over an exposed network. Diffie Hellman lets them combine private exponents with public values to arrive at a shared secret. It establishes keying material; it does not itself encrypt application messages or prove the other party's identity.

13.2 Protocol

Agree on a suitable group, prime parameter p in the familiar modular example, and generator g. Alice chooses private exponent a, calculates A = g^a mod p, and sends A. Bob chooses private b, calculates B = g^b mod p, and sends B. Alice computes K = B^a mod p. Bob computes K = A^b mod p.

```
Alice                                   Bob
secret a                                secret b
A = g^a mod p          --- A --->        receives A
receives B             <--- B ---        B = g^b mod p
K = B^a mod p                           K = A^b mod p
```

The attacker may see p, g, A, and B. The private a and b are not transmitted. In practice, validate received values and group parameters, then derive keys from the shared secret using a KDF and relevant context. A raw group element is not simply used as arbitrary AES key bytes.

13.3 Slide example with every reduction

Let p = 23 and g = 5. Alice chooses a = 6; Bob chooses b = 15.

```
5^2 mod 23 = 2
5^4 mod 23 = 4
A = 5^6 = 5^4*5^2 mod 23 = 4*2 = 8
5^8 mod 23 = 16
B = 5^15 = 16*4*2*5 mod 23 = 19
```

Alice: 19^2 mod 23 = 16 and 19^4 mod 23 = 3, so K = 19^6 mod 23 = 3 times 16 mod 23 = **2**.

Bob: 8^2 mod 23 = 18; 8^4 mod 23 = 2; 8^8 mod 23 = 4. Therefore K = 8^15 mod 23 = 4 times 2 times 18 times 8 mod 23 = **2**.

Both obtain the same value without sending it. These tiny parameters offer no security, since an observer can enumerate exponents.

13.4 Correctness proof

Alice obtains (g^b)^a = g^(ba), and Bob obtains (g^a)^b = g^(ab), all modulo p. Since ab = ba, the results agree. This is a correctness theorem. It does not establish that the observer cannot compute the same secret or that Alice really communicated with Bob.

13.5 DLP CDH and the reduction direction

Discrete logarithm problem or DLP: given g and A = g^a, recover a. **Computational Diffie Hellman or CDH:** given g, A = g^a, and B = g^b, compute g^(ab). The CDH assumption says that this shared value is infeasible for an efficient attacker to calculate in the chosen group and parameter distribution.

If you can solve DLP, recover a from A, then compute B^a. Hence **CDH reduces to DLP**, written CDH <= DLP. DLP solving is sufficient to solve CDH. It does not prove CDH is at least as hard as DLP. The number theory text by Victor Shoup explicitly warns that discrete log hardness alone is not the complete assumption for DH security.

For a full cryptographic key exchange, indistinguishability, authentication, validation, and derivation are additional concerns. Do not replace them all with the single sentence that discrete logarithms are hard.

13.6 Man in the middle attack

An active attacker Mallory intercepts A and B and replaces them with her own public values. Alice obtains a secret shared with Mallory, believing it is shared with Bob. Bob obtains a different secret shared with Mallory, believing it is shared with Alice. Mallory can decrypt incoming protected traffic with one secret and reencrypt it with the other.

The DH equations still work. The failure is lack of authentication of the exchange. An authenticated method can bind the parties, public values, and session context, for example using a pre shared authentication key or verified signatures. PKI mechanisms are excluded; section 11.8 provides only the optional certificate concept.

The class MITM flow (official notes pages 30 and 38):

```
Alice                    Mallory                     Bob
X = g^a  --------------> intercept
                         Z = g^z -------------------> receives Z
receives Z <------------ Z = g^z
                         intercept <---------------- Y = g^b

Alice computes Z^a = g^(az)
Bob   computes Z^b = g^(bz)
Mallory computes X^z and Y^z, matching both secrets
```

Mallory relays data by decrypting under one key and encrypting under the other. Alice and Bob do not share the same secret here. No discrete logarithm has been solved: the attacker chose z. Authenticating the public DH shares, identities, and relevant exchange context prevents this substitution in a properly designed protocol. Keep the honest formulas straight: Alice uses Y^a, not X^a; Bob uses X^b, not Y^b.

13.7 Forward secrecy as a small extension

If each session uses fresh private DH exponents and they are erased after use, later compromise of a long term authentication key need not reveal past recorded sessions. This property is **forward secrecy**, under the protocol's assumptions. Reusing static secret exponents or preserving session secrets changes that protection.

13.8 Checkpoint

Question: What is public in DH, and what remains secret? **Answer:** group parameters and A and B are public; a and b remain private; the resulting shared value should be secret from an observer. **Question:** Does equality of Alice's and Bob's computed values stop Mallory? **Answer:** no; correctness and identity authentication are different guarantees.

14 Authentication and strong authentication

Core. Sources: Introduction to Cryptography slides 19 to 21 and the study documents; Preliminaries slides 44 and 45; the login question in CS- Answers.pdf.

14.1 Identification authentication authorization

Identification states a claimed identity, such as a username. **Authentication** checks evidence supporting that claim. **Authorization** decides what the resulting identity may do. An account may authenticate successfully while still lacking permission to read a particular file.

Authentication can concern a person, device, service, or message origin. Possessing a password proves knowledge of a secret associated with an account under the system's assumptions. It does not prove who physically typed it or that the account is uncompromised.

14.2 Factors

Knowledge: a password or PIN. **Possession:** a security device, smart card, or cryptographic authenticator. **Inherence:** a biometric property. Multi factor authentication combines independent factors from more than one category.

A password and another PIN are two secrets in the same knowledge category, not two independent factor types. A password plus a code generated on a protected token is typically knowledge plus possession. A biometric unlocking a hardware authenticator can contribute to a combined possession and inherence design, depending on how the system implements verification.

Strong authentication requires the actual protocol to protect its factors and checks. MFA is a major improvement over a password alone, but not every MFA method gives equal assurance. An attacker may trick a user into revealing an OTP, compromise the device, or steal an already authenticated session.

14.3 Password threat categories

Brute force tries the space of possible passwords. **Dictionary guessing** tries likely words and variations. **Credential stuffing** reuses username and password pairs leaked elsewhere. **Password spraying** tries a few common passwords across many accounts. **Phishing** deceives the user into revealing credentials or authorizing an attacker operation.

Online guessing interacts with the real service and can be slowed or detected. Offline guessing evaluates candidates against a stolen verifier without the service's rate limits. This is why both service protections and expensive salted password hashing matter.

14.4 Authentication controls linked to the threat

Use unique, sufficiently unpredictable secrets and proper password hashing. Protect communication so passwords are not exposed in transit. Add suitable MFA. Limit repeated attempts per account and other relevant context, with careful monitoring. Give generic failure responses to reduce username disclosure. Secure password reset and account recovery as carefully as normal login.

Account lockout can itself be abused to deny service. Prefer policies that balance attack resistance with availability, such as temporary delays, risk based checks, and safe recovery. Do not give a universal number of allowed attempts without the system's requirements.

Exam answer to login brute force: immediately slow repeated failures, investigate and contain abusive sources, protect affected accounts, and monitor success after suspicious attempts. Long term, strengthen

secrets, MFA, verifier storage, recovery, and testing of bypasses. Merely blocking one IP is insufficient against distributed attempts.

14.5 Challenge response and freshness

A basic shared key illustration is:

```
Client tells server the claimed identity
Server sends fresh nonce N
Client sends T = HMAC_K(identity || server || N || purpose)
Server verifies T and accepts N only once
```

The secret K need not be transmitted in the response. The challenge ties the result to this attempt. Include roles and context to reduce the chance that a response from one use is accepted in another. Mutual authentication requires appropriate evidence from both sides.

This is an illustration for a high entropy shared key. If K is directly derived from a weak password, an observed nonce and response may permit offline password guessing. Do not claim this simple formula is a fully secure password authentication protocol. Real password protocols require more careful design or a protected channel.

14.6 Nonces versus timestamps

A **nonce** is used to distinguish an exchange and, when needed, make a challenge unpredictable. The verifier remembers pending or recently used nonces and rejects reuse. It does not need synchronized clocks merely to check a challenge it issued.

A **timestamp** permits a freshness window but depends on trusted time and acceptable clock differences. An old message can be rejected outside the window. Replay inside the window still needs prevention where harmful. A timestamp and a nonce solve related but different operational problems.

14.7 Biometrics and session limits

Biometrics require matching thresholds and can have false acceptances and false rejections. Unlike a password, a person's physical feature is difficult to replace after compromise. Template protection, liveness and device trust may matter.

Successful authentication usually creates a session. The service must protect the session and recheck authorization for each sensitive request. Stealing a session can bypass the login procedure, so MFA alone does not eliminate every access risk. Detailed cookie and CSRF mechanisms belong to Unit 3 and are not required here.

Checkpoint: Does an OTP prove a user is authorized to edit another account's profile? **Answer:** no; it contributes authentication evidence. The server still needs an object specific authorization check.

15 Authorization access control and privilege

Core. Sources: Introduction to Cryptography slides 22 to 30 and both study documents; the unauthorized profile change scenario in CS- Answers.pdf.

15.1 Policy and enforcement

Authorization is the decision about allowed actions. Access control is the mechanism enforcing that decision. The question is: **may this subject perform this action on this object in this context?** A subject can be a user or process; an object can be a file, record, account, or service; an action can be read, modify, delete, execute, or administer.

For a portal, the policy says a student may read their own result. The server receives a requested record identifier and checks whether that object belongs to the authenticated student. Hiding the link to other students' results is not sufficient enforcement.

15.2 Mandatory Access Control

In **Mandatory Access Control**, a central policy governs access, often through subject clearances and object classifications. Individual owners cannot simply override that policy. A subject's clearance and categories must satisfy the rule for the requested operation.

An example is classified records with labels such as Confidential, Secret, and Top Secret. A user cannot authorize a lower cleared colleague to read a Top Secret document just because the user owns a copy.

Supporting precision: in the Bell LaPadula confidentiality model, the usual rules are no read up and no write down, with categories as well as levels where applicable. The write rule prevents a high classification subject from copying secrets into a low classification object. This is one particular model, not a definition of every mandatory system.

Mandatory policy constrains discretion, but it does not make a system immune to bugs, stolen high clearance accounts, or incorrect policy. The slide's very low risk claim should be read as a design comparison, not a guarantee.

15.3 Discretionary Access Control

In **Discretionary Access Control**, the resource owner can grant or revoke access within the surrounding system's rules. Access control lists and file permission mechanisms are common examples.

The permission string **rwxr--r--** gives the owner read, write, and execute, the group read only, and others read only. Its usual numeric form is **744**. The string **rwxr-xr-x** is **755**. Do not mix these examples.

DAC is flexible and convenient for sharing. It also permits mistaken over sharing and delegation that central confidentiality requirements may forbid. A process running under a user's authority may access what that user can access, which makes malicious code a concern.

15.4 Role Based Access Control

In **RBAC**, permissions are assigned to roles, and users are assigned roles. A Student role permits viewing one's own marks; a Faculty role permits entering marks for assigned classes; an Administrator role permits selected management tasks.

The mappings are users to roles and roles to permissions. When someone changes job, update the role assignment instead of revising many separate object grants. Large systems often use role hierarchies and restrictions on incompatible roles.

RBAC still needs object and context checks. Being Faculty does not necessarily permit editing every department's grades. The role may confer a kind of action, while ownership, assignment, or policy further narrows the allowed objects.

15.5 Compare models

Feature	Mandatory access control	Discretionary access control	RBAC
Main authority	Central policy	Resource owner	Organizational role policy
Decision basis	Labels clearances and policy	Ownership and explicit grants	Role permissions plus constraints
Sharing flexibility	Restricted by mandatory rules	Owner discretion within system limits	Through approved roles
Useful example	Classified documents	Personal file sharing	University or enterprise jobs
Important risk	Wrong labels policy or implementation	Excess grants and unsafe delegation	Excess roles stale assignments and missing object checks

These models can coexist. A hospital could use roles for doctors and nurses, restrict records to assigned patients, and apply mandatory rules for unusually sensitive data. Do not assume a real organization must choose exactly one model.

15.6 Privileges and least privilege

A **privilege** is a permitted action or capability, such as reading a record, changing a setting, executing a program, or managing users. **Least privilege** grants only what is needed for the current task, for the necessary duration and scope.

A service that reads schedules does not need permission to delete the entire student database. A teaching assistant who enters attendance does not automatically need administrator privileges. Removing unused rights reduces the consequences of mistakes and compromised accounts.

Separation of duties assigns sensitive stages to different actors, such as one person proposing a payment and another approving it. **Default deny** rejects actions unless permitted. **Complete mediation** checks each relevant access rather than relying on a one time UI decision.

15.7 Vertical and horizontal escalation

Vertical escalation gives a subject a more powerful role or capability than permitted. A student obtaining administrator functions is vertical. A normal process obtaining system administrator rights is vertical.

Horizontal escalation accesses another subject's objects or functions at a similar privilege level. One student reading another student's private results is horizontal. The attacker may remain a Student but cross an ownership boundary.

In both cases, the violation is access beyond authorization. More login factors do not repair a missing permission check.

15.8 Worked profile scenario

The supplied 2025 answer collection describes changing multiple students' profile emails without authenticating as those students. The likely category is **broken access control**, such as missing object ownership enforcement. If the attacker modifies an object identifier to reach someone else's record, that is often called an insecure direct object reference.

If a logged in student can update another student's email, it is horizontal access escalation. If a low privilege user can call an administrator endpoint, it is vertical. If the scenario never says the attacker was logged in at all, state that missing authentication may also be involved; do not invent a successful login.

Mitigation: authenticate the requester, derive identity from trusted server state, authorize the action on the requested record, reject arbitrary client claimed roles, apply least privilege, log sensitive changes, and test cross account requests. Changing recovery email should receive protection proportionate to its account takeover consequences.

15.9 Authorization is not just a screen

Removing an administrator button is useful interface design, but the server must reject unauthorized direct requests. Signing a role token can protect the token from alteration, but a valid token may still claim permissions that are too broad. Token verification and the actual authorization decision are separate steps.

Checkpoint: A Student role can read any grade record because the endpoint checks only that the role is Student. What is wrong? **Answer:** the role check omits ownership; the endpoint must also restrict the object to the student's authorized records.

16 Put the mechanisms together

Core consolidation. Use this chapter to answer design and scenario questions without assigning every job to encryption.

16.1 Secure submission of an exam document

Authenticate the student using a suitable login mechanism. Authorize submission for the correct account and course. Protect the network exchange using an authenticated encrypted channel. Store the document with appropriate access restrictions and encryption at rest. Use a trusted hash or signed acknowledgment if the workflow needs evidence of the exact file. Keep backups and a recovery process to preserve availability.

A digital signature can provide evidence of content approval when that is required. The service should bind any acceptance evidence to the file digest, submission identity, course, and transaction identifier. A bare hash copied beside the file does not prove who submitted it.

16.2 Which mechanism should I choose

Hide data from an observer: encryption. **Detect attacker modification between shared key holders:** a MAC or authenticated encryption. **Allow third parties to verify a signer's approval:** a digital signature. **Agree on a session secret over a public network:** authenticated key establishment. **Check a person's account claim:** authentication. **Restrict what the account can do:** authorization and access control. **Keep service running:** resilient infrastructure and response processes.

16.3 Packet capture and confidentiality

An observer can capture an encrypted packet without reading its protected content. Capture and decryption are different abilities. Under the assumed secure channel, intact endpoint protection, and uncompromised secrets, the application plaintext remains confidential.

Known plaintext may reveal a portion of the keystream in an XOR construction, but a secure generator is designed so this does not reveal its key or unrelated keystream. Reusing that same mask makes the known portion dangerous for another message. This links the C1 HTTPS question and the 2025 stream question to the same security reasoning.

Half mark HTTP versus HTTPS answer, September 2025 Part B Q7: HTTP is the application request and response protocol; HTTPS carries HTTP over a connection secured using TLS, providing transport confidentiality and integrity and, with proper identity verification, server authentication. Typical default ports are 80 and 443 respectively. Port numbers are not what supplies security. HTTPS does not by itself prevent phishing, weak login logic, or incorrect permissions. See RFC 9110 for the distinction. Detailed TLS handshake and PKI procedures remain outside this module's requested scope.

16.4 How to write a complete algorithm answer

State the purpose and input assumptions. Give setup and key information. Describe the steps or equations. Explain why honest use produces the expected result. State the attacker's available information and the separate security assumption. Mention the most important limitation, such as raw RSA padding or DH authentication.

For comparisons, choose matched dimensions: keys, speed, distribution, security service, structure, and a use case. For scenarios, write cause, consequence, and control in connected statements. For calculations, show intermediate residues and verify the result.

17 Extensions from the supplied slides and older papers

Extension. This chapter accounts for material beyond the written Unit 1 and 2 topic list. Learn it after the core. Its presence in your lecture slides means it can be useful supporting study, but the syllabus does not explicitly require full ECC, ElGamal, RC4, or lattice calculations.

17.1 Random Oracle Model

An ideal random oracle gives an n bit output for every queried input. It is a theoretical model for reasoning about protocols, not a literal implementation of SHA 256.

The three essential properties for September 2025 Part B Q2:

1. **Consistency:** repeated queries on the same input return the same answer.

2. **Uniformity:** the answer to a previously unqueried input is uniformly distributed across the n bit output space.

3. **Independence and unpredictability for fresh inputs:** answers at distinct inputs are independent. Previous answers do not predict a fresh answer before it is queried. Anyone may query the same oracle.

How to imagine it: maintain an initially empty lookup table. On query x, return the stored H(x) if present. Otherwise sample a uniform n bit value, store it at x, and return it. Two distinct inputs can still get the same answer by chance; independence does not mean uniqueness.

Importance, for Part B Q3: the idealization lets a security proof reason about hash queries and exclude exploitable internal structure. It helps relate an attack on a complete construction to its underlying assumption and quantify an attacker's success. It does not by itself make an insecure protocol secure. For a one mark answer, say that ROM treats a hash as a consistent random function to simplify rigorous analysis of cryptographic constructions.

The original random oracle methodology paper describes this proof approach. A proof under the idealization and security after substitution of a real hash are separate claims.

Class wording, official notes page 37: random, independent, and deterministic correspond to uniform fresh outputs, independence across distinct inputs, and consistency on repeated inputs. Deterministic here means the same chosen random function answers the same query consistently. It does not mean all outputs were generated by a short public PRNG seed. On page 36, H: {0,1}* -> {0,1}^l means arbitrary finite bit strings are mapped to l bit strings. The star denotes arbitrary finite length here, whereas Z_n* denotes multiplicative units.

17.2 Double encryption and group closure

Composing two encryption permutations does not automatically double security. If a cipher family is closed under composition, E_K2(E_K1(M)) may equal E_K3(M) for another key K3. Then the effective transformation remains within the same family.

Simple example: a shift cipher over 26 letters with key k maps x to x+k mod 26. Applying shifts k1 and k2 gives x+k1+k2 mod 26, equivalent to a single shift. Even without closure, meet in the middle attacks can undermine naive assumptions about double encryption's strength. Do not generalize the shift example into a claim that every pair of modern encryptions collapses to one key.

17.3 Elliptic curve foundations

For a prime field with p greater than 3 in the common short Weierstrass form, a curve is y^2 = x^3 + ax + b mod p, with 4a^3 + 27b^2 not equal to 0 mod p. Include the point at infinity O. Curve points form an Abelian group under a defined addition operation. O is the identity; the inverse of (x,y) is (x,-y mod p).

For distinct P = (x1,y1) and Q = (x2,y2) with x1 not equal x2, calculate:

```
lambda = (y2-y1) * (x2-x1)^(-1) mod p
x3 = lambda^2 - x1 - x2 mod p
y3 = lambda*(x1-x3) - y1 mod p
```

For point doubling, use lambda = (3*x1^2+a)*(2*y1)^(-1) mod p. If Q is minus P, the sum is O; if doubling a point with y = 0, the result is O. Division must use modular inverses.

Before calculating, check that both points satisfy the curve equation. Chapter 18 checks and solves the specific pair given in Quiz_20, with every modular division shown.

Connecting geometry to the group, official notes pages 10 to 12 and 28 to 29. In the real curve picture, a line through P and Q meets the curve a third time; reflect that third point across the x axis to obtain P+Q. For doubling, use the tangent instead. Over a finite field, the modular formulas implement the algebraic rule; a real plane sketch is only intuition.

The points on a nonsingular curve, together with the point at infinity O, form an Abelian group. O is the identity, and -(x,y) = (x,-y mod p). Thus P+(-P) = O. A point with y = 0 equals its negative, so doubling it gives O rather than attempting to invert zero in the tangent formula. Closure keeps sums on the curve; commutativity follows from the symmetric addition rule. Associativity is a substantive algebraic property of the curve law, not established just by drawing a symmetric line.

For short Weierstrass curves over a prime field of characteristic greater than 3, require 4a^3+27b^2 != 0 mod p. **Singular and supersingular are different concepts:** a singular curve fails this nonsingularity condition and is unsuitable for this group construction. A supersingular elliptic curve is still nonsingular and has a different advanced arithmetic property. You need not derive supersingularity or pairing methods for the requested syllabus.

17.4 ECC security and ECDH

Scalar multiplication kP means repeated group addition, efficiently implemented with doubling and addition. The elliptic curve discrete logarithm problem asks for k given P and Q = kP. For appropriately chosen curves and groups, no efficient general classical attack is known. Special weak curves and incorrect validation can change security.

ECDH uses A = aG and B = bG. Alice calculates aB, Bob bA, and both obtain abG. The same lack of identity authentication as plain DH remains. A roughly 256 bit appropriate elliptic curve group can target approximately 128 bit generic attack security; do not equate group representation length with symmetric key security.

The toy finite curve in the slides, y^2 = x^3 + 2x + 3 mod 17, has 21 affine points plus O, for 22 total points. Its point P = (2,7) satisfies the equation. The real curve illustration is for geometric intuition; finite field calculations should use the modular formulas above rather than reading approximate coordinates from the picture.

Keep two moduli separate, official notes pages 30 and 31. Curve coordinates are calculated modulo the field prime p. Private scalars are chosen according to the order r of the selected base point G, often a large prime subgroup order. These are not generally the same number. Alice sends A = aG; Bob sends B

= bG. The shared point is aB = bA = abG, from which a protocol derives keys. ECDLP asks for a from G and aG; computational ECDH asks for abG from G, aG, and bG. A solver for ECDLP therefore gives a solver for ECDH, which fixes the reduction direction.

17.5 ElGamal encryption

Choose a suitable cyclic multiplicative group, represented here modulo prime p, generator g, and secret s. Publish u = g^s mod p. For message m in the chosen group, choose fresh random r and compute c1 = g^r, c2 = m*u^r, all mod p. Decrypt m = c2*(c1^s)^(-1) mod p.

Correctness: c1^s = g^(rs) = u^r. Multiplication by its inverse cancels the masking value and leaves m. This is a clean DH based structure.

Fresh r is crucial. Reuse under the same public key makes c1 repeat and reuses the multiplicative mask; ratios of c2 values expose ratios of plaintexts. Raw ElGamal is malleable because changing c2 to t*c2 changes the recovered m to t*m. Authentication or an appropriate complete encryption scheme is needed.

Recovering the masking value relates to CDH. Standard indistinguishability of the basic ElGamal construction is usually justified under a decisional DH assumption in a suitable group, which is stronger than merely saying DLP is hard. The 2024 paper's soundness hint is not a complete modern chosen ciphertext security proof.

17.5.1 Elliptic curve ElGamal and its DH connection

Extension, newly explicit in September 2025 Part B Q12. Replace the multiplicative notation of ordinary ElGamal by point addition in a suitable elliptic curve subgroup generated by G. Scalar multiplication takes the role of exponentiation. Encode the message as a point M in this teaching version; arbitrary text is not automatically a curve point.

```
Private key: d                  Public key: Q = dG
Fresh random scalar: r
Encrypt: C1 = rG,  C2 = M + rQ
Decrypt: M = C2 - dC1
Correctness: dC1 = d(rG) = r(dG) = rQ
```

The confidential mask rQ is precisely the elliptic curve DH shared point for Q = dG and C1 = rG. Computing d or r separately is unnecessary if an attacker can compute that shared point directly. The Handbook of Applied Cryptography, Chapter 8 gives the general group formulation and security caveats for ElGamal. Section 18.29 supplies the exam reduction in both directions.

This explains the textbook message recovery connection to computational DH. It does not prove every confidentiality notion or give chosen ciphertext security. The basic point construction is malleable: replacing C2 by C2 + T changes the recovered message to M + T. Practical ECC encryption commonly uses a KEM and authenticated symmetric encryption rather than directly mapping an entire document to curve points.

17.6 RC4 outline

RC4 is a historical stream cipher. Initialize a 256 entry state array S with the values 0 through 255. Its key scheduling algorithm repeatedly updates an index using state and key bytes and swaps array entries. Its generation algorithm continues index updates and swaps, then uses a state based lookup to emit each keystream byte. Encryption XORs each data byte with its corresponding keystream byte.

```
KSA outline:
S[i] = i for i from 0 to 255
j = 0
for i from 0 to 255:
    j = (j + S[i] + key[i mod key_length]) mod 256
    swap S[i] and S[j]

PRGA outline:
i = 0; j = 0
repeat for each byte:
    i = (i+1) mod 256
    j = (j+S[i]) mod 256
    swap S[i] and S[j]
    z = S[(S[i]+S[j]) mod 256]
    output_byte = input_byte XOR z
```

RC4 has statistical weaknesses and is not suitable for new secure systems. It does not define a general modern nonce interface by itself; older protocols combined IVs and keys in their own ways. When an old paper says RC4 IV, understand the enclosing protocol context. Known plaintext plus ciphertext reveals those mask bytes regardless of statistical attacks, as shown in Chapter 7.

17.7 Quantum threat

The supplied Unit 2 Slides (1).pptx is a complete post quantum deck, despite its generic filename. Quantum computation does not accelerate every task. The two main algorithms in these slides are **Shor's**, for factoring and discrete logarithms on sufficiently capable quantum hardware, and **Grover's**, for unstructured search.

Shor's algorithm threatens RSA, DH, and elliptic curve schemes because it attacks their underlying mathematical problems. Increasing an RSA key length does not restore the same kind of asymptotic defense against that algorithm.

Grover's search changes an ideal n bit exhaustive key search from roughly 2^n classical trials to roughly 2^(n/2) quantum oracle queries. The slides describe AES 128 as having a nominal 64 bit quantum search exponent. This is **not an algorithm called AES 64**, and it is not a prediction that a practical quantum machine immediately breaks AES. Circuit cost, error correction, parallelism, and available hardware matter.

Likewise, SHA 256's target preimage search has a nominal quantum query exponent of 128. Quantum collision search is a different problem and should not simply reuse the preimage formula. Learn the broken versus weakened distinction, with the practical assumptions stated.

17.8 Harvest now decrypt later and PQC

An attacker can save encrypted traffic now and try to decrypt it when stronger capabilities arrive. Data requiring secrecy for many years is sensitive to this strategy. The urgency depends on data lifetime and migration time, not merely the date a quantum computer might exist.

Post quantum cryptography or PQC runs on classical computers and is designed around problems believed resistant to both classical and quantum attackers. **Quantum cryptography**, such as quantum key distribution, uses physical quantum effects in the protocol. These terms describe different approaches.

Your deck traces NIST's process from its 2016 call through 2022 selections and the 2024 standards. The key identification for this module is: **ML KEM** for key encapsulation, **ML DSA** for signatures, and **SLH DSA** for stateless hash based signatures. Standards evolve; the module teaches these roles without assuming every candidate named in a slide has a final standard.

17.9 Lattices SVP and CVP

A lattice is the set of all integer linear combinations of independent basis vectors. In two dimensions, basis (2,0) and (0,3) produces points (2m,3n) for integers m and n. Different bases can describe the same lattice. Real cryptographic settings have far higher dimensions.

The **Shortest Vector Problem or SVP** asks for a shortest nonzero lattice vector. The **Closest Vector Problem or CVP** asks for a lattice point nearest a supplied target. CVP with target zero simply returns zero when zero is allowed, so it is not literally SVP. That slide analogy needs this correction.

Short nearly orthogonal bases can make some tasks easier than long skew bases. The good basis versus bad basis intuition motivates some lattice trapdoors, but it does not describe every modern lattice scheme's private key. ML KEM should not be reduced to a hidden good basis explanation.

17.10 Learning With Errors

In LWE, public samples satisfy b_i = inner_product(a_i,s) + e_i mod q, where s is a secret vector and e_i is a small random error. Without errors, enough independent equations can reveal s through linear algebra. Properly chosen noisy high dimensional distributions make recovery difficult under studied lattice related assumptions.

The slide's toy parameters are q = 17 and s = (3,1). For a = (2,5) and e = 1, b = 2*3 + 5*1 + 1 = 12. For a = (4,1) and e = -2, b = 11. For a = (1,6) and e = 0, b = 9. These tiny examples explain the equation and provide no cryptographic security.

An encryption idea combines samples and encodes a bit near 0 or q/2. The secret permits cancellation of the main linear term, leaving small noise around the encoded value. Rounding recovers the bit when parameters and noise bounds are suitable. This is an intuition for correctness, not the full ML KEM algorithm.

Hardness connections concern particular lattice problems, approximation factors, parameters, and reductions. Do not claim that arbitrary noise makes every linear system provably secure, or that absence of a known attack proves universal quantum resistance.

17.11 KEMs signatures and other families

A **Key Encapsulation Mechanism** has key generation, encapsulation, and decapsulation. Encapsulation with the public key returns a ciphertext and shared secret; decapsulation with the private key recovers the secret. A KDF and symmetric protection then use that secret. FIPS 203 specifies ML KEM, derived from the Kyber line of work and associated with Module LWE.

ML DSA, associated with Dilithium, supplies signing and verification rather than shared secret establishment. Its specification is FIPS 204. **SLH DSA**, associated with SPHINCS+, uses a stateless hash based signature construction and is specified by FIPS 205.

Other families in your slides include code based schemes, relying on decoding problems; multivariate schemes, relying on polynomial systems; and isogeny approaches, involving relationships between curves. The slides note that Rainbow and SIKE suffered classical breaks. An algorithm's category label does not by itself establish security.

Two meanings of hybrid: asymmetric plus symmetric protection combines roles and performance; classical plus post quantum establishment combines mathematical assumptions during migration. State which one you mean.

Names in the official notes (pages 14 and 25). Kyber corresponds to the ML KEM key encapsulation line; Dilithium corresponds to ML DSA signatures. They have different jobs: Dilithium does not establish a shared secret merely because it is public key cryptography. Falcon is another lattice based digital signature design, identified by NIST's PQC project. SPHINCS+ is the hash based design underlying SLH DSA. Recognize the role and broad family; do not derive these algorithms. Shortest vector problems belong to lattice mathematics rather than constituting an unrelated additional family.

17.12 Extension checkpoint

Explain why fresh ElGamal r is needed, why ECDH still needs authentication, why PQC runs on an ordinary computer, and why CVP at zero is not the same as finding a shortest nonzero vector. If time is limited, prioritize the core RSA and DH equations before these extensions.

18 Solved questions from your archive and new paper

These are selected actual questions or tightly identified subquestions, with corrected worked solutions. A problem's presence here means it is useful learning material, not a promise that the topic will appear again. Core and supporting questions come first. Extensions are explicitly labeled.

18.1 RSA private exponent from Quiz_20 question 7

Question: selected primes are 13 and 17, public exponent is 35. Find the private exponent.

n = 13 times 17 = 221. phi(n) = 12 times 16 = 192. We require 35d = 1 mod 192. Euclid gives 192 = 5 times 35 + 17, then 35 = 2 times 17 + 1. Back substitution gives 1 = 11 times 35 minus 2 times 192. Therefore **d = 11**.

Check: 35 times 11 = 385 = 2 times 192 + 1. Public key is (221,35), and the corresponding private exponent is 11. A public exponent alone is not the complete public key; the modulus is also needed.

18.2 RSA signature from Quiz_20 question 4

Question: m = 2, p = 3, q = 5, e = 7. Find d and signature S.

n = 15 and phi(n) = 8. Since 7 times 7 = 49 = 1 mod 8, **d = 7**. Compute S = 2^7 mod 15. We have 2^4 = 1 mod 15, so 2^7 = 2^4 times 2^2 times 2 = 1 times 4 times 2 = **8**.

Verify: 8^2 mod 15 = 4 and 8^4 mod 15 = 1. Thus 8^7 mod 15 = 1 times 4 times 8 mod 15 = **2**, matching m. This is the raw representative calculation the paper asks for; a real signature signs a properly encoded digest.

18.3 Euler calculation from the 2025 answer collection

The scan's final handwritten solution makes the intended exponent readable as **23^62 mod 33**. One OCR extraction omitted the exponent, so this solution uses the legible handwritten expression.

33 = 3 times 11, phi(33) = 2 times 10 = 20, and gcd(23,33) = 1. Since 62 = 3 times 20 + 2:

```
23^62 mod 33 = (23^20)^3 * 23^2 mod 33
             = 1 * 529 mod 33
             = 1
```

An even shorter independent check is 23^2 = 1 mod 33, so every even power equals 1. In an Euler question, still state the totient and coprimality to show the requested method.

18.4 Modular fractions from Mid-Sem_16 printed 2018 question 2a

1/5 mod 13: find b with 5b = 1 mod 13. Since 5 times 8 = 40 = 1 mod 13, the answer is **8**.

3/5 mod 7: 5 times 3 = 15 = 1 mod 7, so inverse 5 is 3. Then 3 times 3 = 9 = 2 mod 7. The answer is **2**.

The printed layout stacks the numerator over the denominator. These are modular fractions, not ordinary rational outputs or exponent notation.

18.5 Inverse using Euler from that paper question 2d

Find 7^(-1) mod 15. phi(15) = 8 and gcd(7,15) = 1. By Euler, 7^8 = 1, so inverse is 7^7 mod 15.

7^2 = 49 = 4 mod 15; 7^4 = 16 = 1 mod 15. Thus 7^7 = 7^4 times 7^2 times 7 = 1 times 4 times 7 = 28 = **13 mod 15**. Check 7 times 13 = 91 = 1 mod 15.

18.6 Why the digit sum rule works from question 1b

For decimal number d_k*10^k + ... + d_1*10 + d_0, observe that 10 = 1 mod 3, so every 10^i = 1 mod 3. The number is therefore congruent to d_k + ... + d_0 mod 3.

For 6371, the digit sum is 17, and both leave remainder **2 mod 3**. The same reasoning works modulo 9 because 10 = 1 mod 9. This is a proof using congruence, not a pattern inferred from one example.

18.7 Group subtraction table from question 1d

The given addition table is cyclic with identity a. Associate a,b,c,d with residues 0,1,2,3 modulo 4. Subtraction x minus y means x plus the inverse of y. The inverse pairs are a with a, b with d, c with c, and d with b.

Minus	a	b	c	d
a	a	d	c	b
b	b	a	d	c
c	c	b	a	d
d	d	c	b	a

The first column reproduces the row label because subtracting the identity changes nothing. Every diagonal entry is a because x minus x is the identity.

18.8 Why modular exponentiation is easy from Mid Sem_18 printed 2020 question 5b

Represent the exponent in binary. Repeatedly square the base modulo n and multiply selected powers corresponding to 1 bits. An exponent of bit length t needs at most t squarings and t selected multiplications in a simple implementation. Integer arithmetic is efficient in the bit lengths of the operands.

Give a small example such as 5^13 mod 23 = 21 from Chapter 5. The exponential size of 5^13 as an unreduced integer is irrelevant because you reduce every step. The reverse discrete logarithm problem asks for the exponent and is a different computational task.

18.9 DH reduces to DL from that paper question 7a

Given a CDH instance (g,A = g^a,B = g^b), call a DLP solver to obtain a from A. Then compute B^a = g^(ab). Both the solver call and modular exponentiation are efficient under the assumed efficient DLP algorithm.

Therefore CDH <= DLP. State what is given, what the helper solver returns, and how it produces the required answer. Do not reverse the inequality or say that merely failing to solve DLP proves CDH impossible.

18.10 Easy factoring breaks RSA from question 7b

If an efficient algorithm factors n into p and q, compute phi(n) = (p-1)(q-1), then calculate d = e^(-1) mod phi(n) using extended Euclid. Decrypt any textbook ciphertext with c^d mod n. Thus efficient factoring is sufficient to break RSA private key protection.

This reduction does not show that factoring is necessary for every possible attack. Raw RSA's deterministic or malleable behavior can already defeat stronger confidentiality definitions without factoring.

18.11 RSA chosen ciphertext attack from Mid Sem_19 question 1

The printed 2019 paper defines an IND CCA game for **raw textbook RSA**. The challenged ciphertext is c = m^e mod n, and the adversary may ask for decryption of a different ciphertext.

Choose an invertible r modulo n for which c' = c*r^e mod n differs from c. Query c'. Decryption returns z = m*r mod n. Multiply by r^(-1) mod n to recover m, and determine which challenge message was encrypted.

For an illustration with n = 55, e = 3, m = 7, and c = 13, choose r = 2. c' = 13 times 8 mod 55 = **49**, different from 13. The oracle decrypts 49 to **14**. Since inverse 2 mod 55 = 28, recover 14 times 28 mod 55 = **7**.

There is also a simpler failure of indistinguishability: encryption is public and deterministic, so the attacker can encrypt the two candidate messages and compare. The modified ciphertext attack directly uses the game's CCA capability. Padding and a complete secure scheme address these raw primitive failures.

18.12 Cross set hash collisions from Mid-Sem_16 question 2e

Two independent sets each have k sampled inputs, and the ideal digest has l bits. Let N = 2^l. There are k^2 cross set pairs. Under the usual birthday approximation, probability of at least one match is:

```
P approximately = 1 - exp(-k^2/N)
```

For P no more than one half, exp(-k^2/N) is at least one half, giving k^2/N <= ln 2. Therefore **k <= sqrt(ln 2) * 2^(l/2)**, approximately 0.833*2^(l/2).

This is an approximation in the large output space regime with uniform independent digest behavior. Cross pair events are not all independent, so do not label the exponential formula an exact finite probability. A conservative union bound gives P <= k^2/N, and k <= sqrt(N/2) guarantees P <= one half under the pairwise uniform model.

18.13 Unequal hash sets from End-Sem_18 question 2c

Let |A| = k and |B| = 2k, with m bit ideal digests. The number of cross pairs is 2k^2. The approximation is P = 1 - exp(-2k^2/2^m). Requiring P <= one half gives:

```
2k^2 / 2^m <= ln 2
k <= sqrt(ln 2) * 2^((m-1)/2)
```

This matches the paper's threshold formula. For a strictly less than one half probability, use a strict inequality in the approximate condition. Explain why the factor 2 arises: the second set is twice as large.

18.14 Password lifetime from C1 2023 question 9

There are 26 lowercase letters plus 10 digits, so 36 choices per character. Under independent uniform choice, five characters yield 36^5 = 60,466,176 candidates. At an explicitly assumed rate r guesses per second, worst case time is 60,466,176/r, and average successful exhaustive time is about half that.

At r = 10^6, this is about 60.47 seconds worst case and 30.23 seconds average. Explain whether you mean offline or online guessing. An online service's throttling and monitoring can make the effective rate much smaller. Human chosen patterns can make actual guesses much fewer than the full search space.

18.15 Salt size calculation from Mid-Sem_16 question 3b

The printed question uses 32 bit user IDs, 64 bit passwords, 16 bit digests, and a salt expanded from 4 to 6 bits. Its security gain is a storage comparison, not an increase in cryptographic entropy.

Under the apparent exam model, a precomputed table needs a separate collection for every salt. Moving from 2^4 to 2^6 salts multiplies that dictionary size by 4, a **300 percent increase**, assuming fixed per salt table entries. A verifier record stores ID, digest, and salt: 32+16+4 = 52 bits becomes 54 bits, a **100*(2/52) = 3.846 percent increase**.

The defined ratio is 300/3.846 = **78**, which is around 80 as requested. Explicitly state the storage model. If the dictionary itself stores a salt field in every record, the exact storage ratio changes. In real systems, adequate salt length prevents cross account precomputation reuse; a short digest such as 16 bits is itself unsuitable for a secure password verifier.

18.16 Security objectives from C1 2023 questions 2 to 4

MD5 or an unkeyed hash: intended as an integrity fingerprint when compared with a trusted reference, not confidentiality, sender authenticity, or non repudiation. MD5's broken collision resistance limits adversarial use, so state both the intended category and its current limitation.

RSA encryption scheme: confidentiality when used as a properly constructed scheme under appropriate assumptions. Bare RSA encryption does not establish sender authenticity, integrity, or non repudiation. Anyone can encrypt using the recipient's public key.

RSA digital signatures: integrity, origin authentication, and support for non repudiation, assuming secure encoding, trustworthy public key attribution, and protected private keys. No message confidentiality is supplied by signing alone.

18.17 Can a sniffer read HTTPS from C1 2023 question 5

A sniffer can capture HTTPS traffic. Under a properly established secure connection and uncompromised endpoint secrets, the protected application data remains ciphertext to the observer. Metadata such as addresses, lengths, and timing can still be visible. Decryption becomes possible in different threat models, such as access to exported session secrets, a compromised endpoint, or a deliberately configured intermediary.

Do not state that encryption prevents packets from being captured. This question is about reading captured content, not about whether a packet can be observed at all.

18.18 Three active attacks and insider advantages from C1 questions 8 and 10

Message modification can change an instruction and violate integrity. **Masquerade** can impersonate a legitimate account and misuse its permissions. **Denial of service** can exhaust resources and violate availability. Replay is another valid active example if you explain how it repeats an unauthorized action.

An insider may have valid credentials, physical or internal access, knowledge of systems and procedures, and activity that initially looks ordinary. Least privilege, separation of duties, review of sensitive activity, and revocation of obsolete access reduce these advantages.

18.19 Non repudiation from C1 question 7

Require a valid digital signature tied to the exact message and an authentic public key for the sender. Verify before accepting the action, preserve the message and signature, and bind relevant transaction context so the evidence refers to the intended action.

A MAC cannot independently identify the sender to a third party because the receiver also possesses the generating key. A signature's evidence remains conditional on key custody and attribution; it does not make every possible denial logically impossible.

18.20 DH correctness and assumption from C3 2023 question 4

State the purpose: establish shared keying material. Give p,g,a,b,A,B and the two K computations. Prove B^a = g^(ba) = g^(ab) = A^b. Define the DH problem as computing g^(ab) from g,g^a,g^b, and the CDH assumption as computational infeasibility in the chosen group. Finally identify the need to authenticate the exchange against an active intermediary.

This answer directly covers the paper's purpose, protocol, correctness, and assumption subparts. Detailed PKI is not needed.

18.21 DH man in the middle numerical from End-Sem_18 question 1a

Supporting numerical extension. The printed values are g = 6, p = 13, attacker's sent half key R_c = 2, K_ac = 10, and K_bc = 8. Find the honest K_ab.

Alice computes 2^a = 10 mod 13; enumerate the small toy group and obtain a = 10 mod 12. Bob computes 2^b = 8 mod 13, giving b = 3 mod 12. Their honest public values would be A = 6^10 mod 13 = 4 and B = 6^3 mod 13 = 8.

Then **K_ab = 6^(10*3) mod 13 = 6^30 mod 13 = 12**, because 6 has order 12 and 30 = 6 mod 12. A direct check is B^a = 8^10 mod 13 = 12. This enumeration is feasible only because the numbers are tiny.

18.22 ECC addition and the new subtraction variant

Extension. Curve y^2 = x^3+x+1 mod 13, P = (4,2), Q = (10,6).

Check P: 2^2 = 4, and 4^3+4+1 = 69 = 4 mod 13. Check Q: 6^2 = 36 = 10, and 10^3+10+1 = 1011 = 10 mod 13. Both are valid.

The slope is (6-2)/(10-4) = 4*6^(-1) mod 13. Since 6 times 11 = 66 = 1 mod 13, lambda = 44 mod 13 = 5. Then x_R = 5^2-4-10 = 11 mod 13. y_R = 5*(4-11)-2 = -37 = 2 mod 13. Therefore **P+Q = (11,2)**.

Check the result: 2^2 = 4 and 11^3+11+1 = 1343 = 4 mod 13. This final check catches arithmetic or sign errors.

September 2025 Part B Q10 changes the operation to S - T. It gives S = (4,2), T = (10,7), and the same E(1,1) over F_13. First negate T: -T = (10,-7 mod 13) = (10,6). Thus S - T = S + (-T), exactly the addition calculated above, and **S - T = (11,2)**. Also check T itself: 7^2 = 10 mod 13, matching its curve right side. Do not subtract coordinate pairs directly. This new question reuses the same calculation once the group inverse is applied.

18.23 ElGamal decryption from Mid-Sem_16 question 3a

Extension. In Z_11*, generator g = 2 and public u = 10. Since 2^5 mod 11 = 10, private exponent s = 5 in this toy group.

For (c1,c2) = (6,4): 6^2 = 3, 6^4 = 9, and 6^5 = 10 mod 11. The inverse of 10 is 10 because 10^2 = 1 mod 11. Therefore m = 4 times 10 mod 11 = **7**.

For (5,7): 5^2 = 3, 5^4 = 9, and 5^5 = 1 mod 11. Its inverse is 1, so m = 7 times 1 = **7**. Two different ciphertexts can encrypt the same message because fresh randomness changes the mask.

18.24 ElGamal correctness from the 2024 endsemester image

Extension. With u = g^s, c1 = g^r, and c2 = m*u^r, decrypt c2*(c1^s)^(-1). Since c1^s = g^(rs) = u^r, this equals m*u^r*(u^r)^(-1) = m. State the group so the inverse exists.

The soundness subquestion's hint connects the shared mask to DH. A stronger modern confidentiality argument uses appropriate decisional assumptions and a complete scheme. Raw multiplicative ElGamal remains malleable, so the correctness proof does not establish chosen ciphertext security.

18.25 Known stream byte from the 2025 collection

Extension with core XOR reasoning. The supplied answer gives a known first plaintext byte of 0xAA from a particular frame format. If C = P XOR Z, then **Z = C XOR 0xAA** for that byte. This is a known plaintext calculation, not a statistical attack.

It recovers that keystream byte, not necessarily the RC4 key or all future bytes. The stated plaintext format assumption is essential. With a reused keystream, the recovered byte can reveal the corresponding byte of another message.

18.26 Random versus incremental IV repetition from that collection

Supporting extension. For N bit independently random IVs, the chance of a collision after k draws is approximately 1 - exp(-k*(k-1)/2^(N+1)). Significant probability occurs at order **2^(N/2)** draws, with a 50 percent threshold near 1.177*2^(N/2).

A counter produces distinct IVs for **2^N** values before wraparound if its state never resets and the scope is managed correctly. The first repeat appears on the next value if it wraps under the same key and prefix.

Best, average, and worst cases in draw counts, including the new clear screenshot wording: put M = 2^N. For independent uniform random draws with replacement, the first repeat can occur on draw 2, the expected first repeat is Theta(sqrt(M)) = Theta(2^(N/2)), and the latest possible first repeat is draw M+1 by the pigeonhole principle. Thus best is Theta(1), average Theta(2^(N/2)), and worst Theta(2^N). The expectation has leading approximation sqrt(pi*M/2); it differs from the 50 percent threshold above.

For a fixed N bit counter starting anywhere and incrementing modulo M under unchanged key and prefix, the first repeated value occurs on draw M+1. Best, average, and worst draw counts are all Theta(2^N) under those assumptions. If the system stops or rekeys before wrapping, it avoids that repeat. Reset bugs invalidate the guarantee.

These counts concern waiting until any repeated IV. They are not the cost of generating one IV, and not the probability of hitting one specific earlier IV, which is 1/M per independent random draw. The handwritten archive answer's O(N) and O(N/2) statements confuse bit length with the number of possible IVs.

18.27 Orders and generators from September 2025 Part B Q9

Supporting group mathematics. The group is Z_7* under multiplication modulo 7, with elements 1 through 6 and identity 1. Its size is 6. The order of a is the first positive r for which a^r = 1 mod 7; do not count a^0 as evidence that every order is zero.

Element	Successive powers until the first 1	Order	Generator of Z_7*
1	1	1	No
2	2, 4, 1	3	No
3	3, 2, 6, 4, 5, 1	6	Yes
4	4, 2, 1	3	No

Element	Successive powers until the first 1	Order	Generator of Z_7*
5	5, 4, 6, 2, 3, 1	6	Yes
6	6, 1	2	No

Therefore **3 and 5 are the generators**. Every order divides 6, as required by Lagrange's theorem. Faster generator test: because 6 has prime divisors 2 and 3, check g^(6/2) and g^(6/3). An element generates the whole cyclic group if neither is 1. For 3 these powers are 6 and 2. This test assumes you know the group order and its prime factors.

18.28 Worked quadratic residue definition for September 2025 Part B Q8

Extension illustration, not an additional original paper numerical. Squaring 1 through 6 modulo 7 gives 1, 4, 2, 2, 4, 1. The nonzero square residues are therefore 1, 2, and 4; 3, 5, and 6 are nonsquares. For a = 2, Euler's criterion gives 2^3 = 1 mod 7. For a = 3, it gives 3^3 = 6 = -1 mod 7.

For the composite example n = 77 = 7*11, a = 4 is a square because 2^2 = 4 mod 77. In contrast, a = 13 is 6 modulo 7 and 2 modulo 11, both nonsquares in their respective fields. Therefore 13 cannot be a square modulo 77. Nevertheless its Jacobi symbol is +1 because its two prime factor signs are both -1 and their product is +1. This explains why Jacobi +1 alone is insufficient. Here the factors are deliberately provided for teaching; the cryptographic problem hides a large modulus's factors.

18.29 Elliptic curve ElGamal message recovery and DH from September 2025 Part B Q12

Extension. Use a suitable subgroup with generator G, public key Q = dG, and ciphertext (C1,C2) = (rG, M+rQ). The elliptic curve computational DH problem asks for abG from G, A = aG, and B = bG.

Direction 1: a DH solver breaks message confidentiality. Supply Q and C1 to the DH solver. It returns drG = rQ. Compute M = C2 - drG. Thus computing the shared point permits decryption without learning d or r.

Direction 2: a universal textbook message recovery algorithm solves DH. Given G, A = aG, and B = bG, set the public key to A and construct the ciphertext (B,G). In the textbook group encoding this is a valid encryption of some point M = G-abG using scalar b. If an algorithm can recover the message from every such ciphertext without the private key, its output is M. Return G-M = abG. All points are in the selected subgroup.

```
DH challenge: G, A=aG, B=bG
Run message recovery with public key A and ciphertext (B,G)
Recovered M = G - aB
Return G - M = aB = abG
```

Three mark answer: state the ciphertext equations, identify rQ as the DH shared point, and show the reduction direction required by the claim. For the phrase "as secure as DH," show both directions and explain the assumptions. The second direction assumes recovery on arbitrary valid textbook point ciphertexts; a routine that only succeeds for a special message distribution is a different claim. The algebraic correspondence is not a proof of indistinguishability or chosen ciphertext security, and it does not make raw ElGamal nonmalleable. Appropriate decisional assumptions and a complete secure construction are needed for stronger guarantees.

18.30 Modular powers from official notes pages 12 13 23 and 24

Question A: calculate 5^5 mod 13. Since 5^2 = 25 = -1 mod 13, 5^4 = 1, hence 5^5 = **5**. Recognizing a short cycle saves work.

Question B: calculate 2^100 mod 13. Fermat gives 2^12 = 1 because 13 is prime and 2 is nonzero. Since 100 = 8*12+4, reduce to 2^4 = 16 = **3**. Show the condition and the exponent division, not just the answer.

18.31 Sums of powers from the same official notes

Question A: calculate (1^10+2^10+...+10^10) mod 11. Each of the ten bases is nonzero modulo prime 11, so Fermat makes each term 1. The sum is **10 mod 11**.

Question B: calculate (1^50+2^50+...+5^50) mod 7. Since 50 = 8*6+2, each term reduces to its square. Reduce the squares of 1, 2, 3, 4, 5 to 1, 4, 2, 2, 4 respectively. Their sum is 13 = **6 mod 7**. The upper bound matters: adding 6^50 contributes 1 and gives 0. Some continuation scratch on page 24 includes that sixth term; it is not the answer to the original sum ending at 5.

18.32 Divisibility proof from official notes page 26

Question: prove that 42 divides n^7-n for every integer n.

Since 42 = 2*3*7 with pairwise coprime factors, prove divisibility by each. Modulo 2, n^7 and n have the same parity. Modulo 3, if 3 divides n, both are zero; otherwise n^2 = 1, so n^7 = n*(n^2)^3 = n. Modulo 7, Fermat's all integer form directly gives n^7 = n. Thus the difference is divisible by 2, 3, and 7, hence by **42**.

Common mistake: starting with n^6 = 1 mod 7 without considering multiples of 7. That equation needs n nonzero modulo 7. The form n^7 = n covers both cases. Proving divisibility by factors gives divisibility by their product here because they are coprime; this is not true for arbitrary overlapping factors.

18.33 Inverses from official notes pages 27 and 28

Question A: find 12^(-1) mod 31 using Fermat. Compute 12^29 because 31 is prime. Repeated squaring gives:

```
12^1  = 12 mod 31
12^2  = 20 mod 31
12^4  = 28 mod 31
12^8  =  9 mod 31
12^16 = 19 mod 31
29 = 16+8+4+1
12^29 = 19*9*28*12 = 13 mod 31
Check: 12*13 = 156 = 1 mod 31
```

The answer is **13**. For an independent Euclid check: 31 = 2*12+7; 12 = 7+5; 7 = 5+2; 5 = 2*2+1. Back substitution gives 1 = 13*12-5*31. The handwritten 12^2 residue must be 20, not 9; 9 is the residue of 12^8.

Question B: find 7^(-1) mod 17 using Fermat. The exponent is 15 = 8+4+2+1. The residues are 7^2 = 15 = -2, 7^4 = 4, and 7^8 = 16 = -1. Therefore 7^15 = (-1)*4*(-2)*7 = 56 = **5 mod 17**. Check 7*5 = 35 = 1 mod 17. When an exponent is split into a sum, multiply the corresponding powers; do not add them.

18.34 Square roots from official notes pages 9 10 19 and 20

Question A: list the nonzero squares modulo 11 and their roots. Squaring 1 through 5 suffices because the negatives give the same squares.

Residue a	Two roots modulo 11	Verification
1	1 and 10	1 and 100 both give remainder 1
3	5 and 6	25 and 36 both give remainder 3
4	2 and 9	4 and 81 both give remainder 4
5	4 and 7	16 and 49 both give remainder 5
9	3 and 8	9 and 64 both give remainder 9

Since 11 = 3 mod 4, the shortcut for a = 3 gives 3^3 = 5; negate it to obtain 6. Always check the square.

Question B, an added CRT illustration: find every solution of x^2 = 4 mod 15 when 15 = 3*5 is factored. Modulo 3 the roots are 1 and 2; modulo 5 the roots are 2 and 3. Combine each pair by CRT: (1,2) gives 7; (1,3) gives 13; (2,2) gives 2; (2,3) gives 8. The roots are **2, 7, 8, 13**. Squaring gives 4, 49, 64, 169, each with remainder 4 modulo 15. A composite modulus can have more than two roots, so the prime field rule does not apply unchanged.

18.35 Curve points from official notes pages 11 12 21 and 22

Class curve: y^2 = x^3+x+1 over F_11. Take A = (2,0) and B = (3,3). For A, the right side is 8+2+1 = 11 = 0, matching 0^2. For B, it is 27+3+1 = 31 = 9, matching 3^2. The negative of B is (3,8). There is no point with x = 5: the right side is 125+5+1 = 131 = 10, a nonsquare modulo 11.

For A+B, lambda = (3-0)/(3-2) = 3. Then x_R = lambda^2-2-3 = 9-5 = 4, and y_R = lambda*(2-4)-0 = -6 = 5. Therefore **A+B = (4,5)**. Check y_R^2 = 25 = 3; x_R^3+x_R+1 = 64+4+1 = 69 = 3. The result lies on the curve.

Since A has y = 0, A = -A and **2A = O**. Do not attempt to invert 2y = 0. For an additional doubling check, 2B has lambda = (3*3^2+1)/(2*3) = 28/6 = 6*2 = 1 mod 11. Thus x = 1-6 = 6 and y = (3-6)-3 = 5, so **2B = (6,5)**. These are tiny insecure teaching parameters.

18.36 Polynomial multiplication from official notes page 38

Question: multiply 1010 and 0011 over bit coefficients and reduce modulo x^3+x+1. Translate to (x^3+x)(x+1). Expansion gives x^4+x^3+x^2+x. Subtract, equivalently XOR, x*f(x) = x^4+x^2+x, leaving x^3. Subtract f(x), leaving **x+1**, or **011** in the three bit field representation.

A faster check is to reduce the first input first: x^3+x = (x+1)+x = 1. Then 1*(x+1) = x+1. The class's four bit input is a polynomial awaiting reduction, not evidence that this cubic constructs GF(2^4). The degree of the irreducible modulus is 3, so the field has 8 elements. Section 5.9.1 gives addition, division, and an inverse in this same field.

18.37 RSA with d chosen first from official notes page 32

Use p = 3, q = 7, so n = 21 and phi(n) = 12. Choose d = 5. Its inverse modulo 12 is e = 5 because 25 = 1 mod 12. Public key: (21,5). Private exponent: 5. For the class message m = 20 = -1 mod 21, c = 20^5 = -1 = **20**, and decryption gives 20 again. The equality of message and ciphertext for this representative is legitimate textbook arithmetic, not an error in the formula.

For an added check with a changing representative, take m = 2. Then c = 2^5 = 32 = **11 mod 21**. Since 11^2 = 16, 11^4 = 4, and 11^5 = 44 = 2 mod 21, decryption returns 2. These tiny primes, equal exponents, and raw deterministic encryption are teaching choices, not secure RSA parameter recommendations.

19 Revision and practice

19.1 Minimum memory sheet

Item	Remember
CIA	Confidentiality integrity availability with matched examples
Authentication authorization access control	Verify claim decide permission enforce permission
Pairwise symmetric keys	n(n-1)/2 under distinct pairwise keys
Search space	A^L or 2^b for independent uniform choices
Expected exhaustive trials	About half the space
Modular inverse	a*b = 1 mod n exists iff gcd(a,n)=1
Totient for distinct primes	phi(pq) = (p-1)(q-1)
Euler	Coprime a gives a^phi(n)=1 mod n
RSA	n=pq d=inverse e mod phi encrypt m^e decrypt c^d
RSA signature representative	s=h^d verify s^e=h with secure encoding in real use
DH	A=g^a B=g^b K=B^a=A^b=g^(ab)
DES	64 bit block 56 effective key 16 rounds
AES	128 bit block keys 128 192 256 rounds 10 12 14
AES final round	Omits MixColumns initial AddRoundKey occurs first
MD5	128 bit digest 512 bit blocks 64 operations broken collisions
SHA 256	256 bit digest 512 bit blocks 64 rounds eight 32 bit state words
Hash attacks	Target digest fixed input or freely chosen pair
Ideal hash collision effort	Approximately 2^(n/2)
Shared key MAC	Integrity and authentication not public attribution
Signature	Private sign public verify no secrecy by itself
Nonce reuse in XOR masks	Exposes relationships between plaintexts
Escalation	Vertical gains stronger privilege horizontal crosses peer objects

19.2 Diagram checklist

Reproduce the DES top level flow, a Feistel round, DES F, DES key generation, an AES round sequence, an iterated hash, HMAC nesting, signature verification, and the two party DH exchange. Label widths and key ownership. The archive's diagram sheet is evidence that structures deserve practice, but its SHA 512 diagram must not be mislabeled SHA 256.

19.3 Practice questions before looking at answers

1. A backup disk is stolen but encrypted. Explain confidentiality and availability separately.
2. Distinguish threat, vulnerability, vector, surface, exploit, control, and risk using one example.
3. Give three active attacks and one passive attack, with the relevant security objectives.
4. Explain why a signed document can be public and still useful for security.
5. Compare symmetric and asymmetric cryptography and justify a hybrid scheme.
6. Find inverse 11 mod 26 and calculate 3/7 mod 11.
7. Compute 7^13 mod 19 using repeated squaring.
8. For RSA p=7,q=11,e=7, find d and encrypt m=9.
9. For DH p=23,g=5,a=4,b=3, compute both public values and the shared value.
10. Draw DES and explain why its six to four bit S boxes do not prevent decryption.
11. Draw AES and distinguish block size, key size, rounds, and round keys.
12. A cipher has good avalanche behavior. Has its security been proven? Explain.
13. Distinguish preimage, second preimage, and collision resistance with one attack each.
14. Explain why a file plus its hash can be changed by an active attacker.
15. Explain the roles of HMAC's inner and outer hashes.
16. Compare CBC and CTR in IV requirements, parallelism, padding, and error effects.
17. Two messages reuse a CTR nonce and overlapping counters under the same key. Derive the information leak.
18. Explain why password plus PIN is not two factor authentication.
19. Compare MAC, DAC, and RBAC in the access control sense.
20. A server checks only that a user is logged in before returning a requested salary record. Diagnose and mitigate.
21. Explain why plain DH permits a man in the middle even though the keys are equal for honest parties.
22. Explain why hard factoring is not by itself a proof that raw RSA is IND CCA secure.
23. A random generator passes statistical tests but uses a public seed. Is it safe for keys?
24. An administrator account is compromised but no outage is reported. Give a careful CIA assessment.

19.4 Answers and self marking

1. Confidentiality may hold if the key is protected; access to that physical copy is lost, and availability depends on alternatives. 2. Use the asset to weakness to action to consequence chain from Chapter 2 and separate collection of entry points from the chosen route. 3. Modification, masquerade, replay, or DoS are active; observation or traffic analysis is passive. 4. Public verification establishes integrity and signing evidence without secrecy. 5. Symmetric protects bulk data efficiently; authenticated asymmetric establishment addresses shared key setup.

1. Inverse 11 mod 26 is **19**; inverse 7 mod 11 is 8, so 3/7 is **2 mod 11**. 7. 7^2=11, 7^4=7, 7^8=11 mod 19, giving 7^13 = 11*7*7 = **7 mod 19**. 8. n=77, phi=60, d=43; encryption 9^7 mod 77 = **37**. 9. A=4, B=10, and both shared values are **18**.

1. Feistel reversibility does not require invertible F. 11. All AES blocks are 128 bits; rounds vary by key size and include an initial key addition. 12. Avalanche is a statistical design property, not a proof. 13. Correctly state what is fixed and what the attacker chooses. 14. The attacker can recompute the hash unless the reference is trusted. 15. Key dependent nesting prevents the naive secret prefix length extension issue.

1. CBC needs an unpredictable IV, sequential encryption and usually padding; CTR needs unique inputs, parallel operation and no block padding. Match error effects precisely. 17. XORing ciphertexts cancels the reused mask. 18. Both are knowledge factors. 19. State central labels, owner discretion, and role permissions respectively. 20. Authentication alone omits authorization on the object; check requested action and record scope server side.

1. The attacker establishes two separate correct exchanges. 22. Determinism and malleability already give attacks. 23. No: the attacker can reproduce the key sequence. 24. Describe information access and modification capability, and state availability as potential unless actual disruption is supported.

Award yourself full credit only when the answer includes the reasoning. A correct numeric answer without a check or major steps is not yet exam ready.

19.5 Practice paper made for this module

This is a **new practice paper**, not a recovered past paper. Total **50 marks**. Attempt without the solutions, then compare with the referenced chapters.

1. A compromised administrator reads student records, changes grade rules, and disables login. Analyze CIA, the surface, and immediate controls. **8 marks.**

2. Compare symmetric and asymmetric cryptography and explain why both are used together. Include the pairwise key formula. **5 marks.**

3. For RSA p=13,q=17,e=35, find d; encrypt representative m=2; state the correctness argument and a limitation of raw RSA. **8 marks.**

4. Draw DES's main structure and F, label widths, and summarize the key schedule. Contrast two AES facts. **7 marks.**

5. Define the three hash resistance properties, compare MD5 and SHA 256, and explain why HMAC is needed beyond a bare hash. **7 marks.**

6. State DH, calculate the p=23,g=5,a=6,b=15 example, prove equality, and explain its authentication weakness. **7 marks.**

7. Explain proper PRNG requirements, independent authentication factors, the three access control models, and both types of privilege escalation. **8 marks.**

19.6 Marking guide

1. Three CIA explanations 3, entry point and weakness 2, concrete immediate actions 3. 2. Matched comparison 3, hybrid explanation 1, formula with assumptions 1. 3. n=221 and phi=192 2, d=11 with inverse check 2, **c=59** for m=2 1, correctness 2, raw limitation 1. 4. Overall flow 2, F with widths 2, schedule 1, two AES contrasts 2.

1. Three properties 3, MD5 and SHA 256 comparison 2, active attacker and keyed tag explanation 2. 6. Protocol 2, A=8 B=19 K=2 calculations 2, correctness 1, MITM and authentication need 2. 7. Randomness versus unpredictability 2, independent factors 2, models 2, escalation types with examples 2.

19.7 Flexible study sequence

Session 1: CIA, threats, surfaces, and the two supplied incident scenarios. Produce two written scenario answers. **Session 2:** data states, keys, randomness, XOR, modular inverses, Euler, and repeated squaring. Complete six calculations without notes.

Session 3: symmetric versus asymmetric, hybrid design, DES, and AES. Redraw each main diagram from memory. **Session 4:** hashing, MD5, SHA 256, HMAC, signatures, and the birthday formulas. Explain why bare hashes and shared MACs have different limits.

Session 5: RSA and DH. Solve the archive's signature, inverse, correctness, reduction, and key agreement questions. **Session 6:** authentication, authorization, access models, privilege escalation, and integrated design. Diagnose three unfamiliar permission scenarios.

Session 7: attempt the practice paper, repair weak areas, and only then study selected extensions. If you have fewer days, combine sessions; if you have more, space them out and redo retrieval practice the next day. No exam date was supplied, so this is a sequence rather than a fixed calendar.

19.8 Official class notes retrieval checks

Close the module and answer these before repeating the new worked exercises.

1. Which inverse method works without knowing whether the modulus is prime or knowing its factors?
2. When is a^(p-1) = 1 valid, and when is a^p = a valid?
3. Does an Abelian group necessarily have one generator? What does ord(3) = 5 modulo 11 tell you?
4. What are the nonzero squares modulo 11? Does Euler's criterion itself calculate a root?
5. In GF(2^3), why is multiplication different from arithmetic modulo 8?
6. In the class RSA method, does choosing d first make d public?
7. In ECDH, are the coordinate modulus and scalar subgroup order necessarily equal?
8. Why do a correct signature equation and a collision resistant hash not by themselves prove a secure signature scheme?

Answers: 1. Extended Euclid, provided the gcd is 1. 2. The first needs prime p and p not dividing a; the second needs prime p and holds for every integer a. 3. No; 3 generates a subgroup of size 5, not all ten units. 4. {1,3,4,5,9}; no, it tests membership. 5. It uses polynomial coefficients modulo 2 and an irreducible cubic; every nonzero element has an inverse, unlike Z_8. 6. No; (n,e) remains public and d private. 7. No; coordinates use p while scalars use the selected base point order. 8. Correctness concerns honest behavior; unforgeability concerns adversaries and depends on the full scheme and assumptions.

20 What not to memorize incorrectly

- A key's number of bits is not always its effective security level.
- A larger output from a KDF does not add entropy to a weak password.
- A random looking sequence can still be predictable.
- Correctness is not adversarial security.
- Signing does not encrypt the message.
- A MAC verifier can also generate MACs with the shared key.
- Hash collisions exist mathematically; security concerns finding them efficiently.
- CBC and CTR IV or nonce requirements differ.
- Block cipher use does not always require padding.
- DH does not authenticate identities by itself.
- A valid login does not authorize every object or operation.
- Mandatory access control and message authentication codes share an acronym but not a function.
- Losing encryption keys can destroy availability even when confidentiality is preserved.
- A current syllabus outranks the scope of an older paper.

21 Audit of every supplied file and past paper

This audit is the basis for the priorities. It separates current syllabus coverage from older course requirements. **Core** is explicitly in Units 1 and 2, **supporting** deepens those objectives, **extension** goes beyond their named requirements, and **outside** belongs to other units or older course structures. PKI stays excluded even when an old paper asks for it.

21.1 Teaching material inventory

File	Material read	Treatment in this module
Syllabus.pdf	All 3 pages	Units 1 and 2 define core scope PKI excluded basic certificate concept briefly retained
Introduction to Cyber Security.pptx.pdf	All 55 pages including image based conceptual diagrams	CIA threats principles surfaces metrics incidents and broader context
Unit 2 Intro to Cryptography.pptx	All 32 slides and available speaker notes	Data states seven objectives authentication access and privilege
Unit 2 Preliminaries (1).pptx	All 48 slides and available speaker notes	Arithmetic keys complexity algebra correctness and hybrid methods
Unit 2 Symmetric Crypto (1).pptx	All 45 slides and available speaker notes	Hashes MACs design DES AES stream ideas and modes
Unit 2 Asymmetric Crypto (1).pptx	All 43 slides and available speaker notes	RSA DH hard problems ECC extension PKI read for scope but not taught
Unit 2 Slides (1).pptx	All 25 slides and available speaker notes	Post quantum material in extension Chapter 17
Introduction to Cryptography Study Material (1).docx	Entire document including tables	Matches intro cryptography content reinforces access control
Unit 2 Cryptography Study Material.docx	Entire document checked against other study file	Byte identical to the other DOCX does not add a second unit of content

The teaching decks total **248 pages or slides**, comprising the 55 page PDF and 193 PowerPoint slides. Source headings or notes saying Module 1 are not treated as instructions about your requested exam scope. They are labels inside teaching material.

21.2 ZIP file 1 nse quiz

nse-quiz (1).pdf, 4 pages. The first three contain ten multiple choice questions; the fourth is blank. Q1 concerns host firewalls; Q2 and Q8 concern Snort; Q3 IP spoofing; Q4 firewall profiles; Q5 SYN floods; Q6 indicators of compromise; Q9 sandbox detection; Q10 scan responses. These are outside the detailed Unit 1 and 2 core, mostly Units 3 and 4.

Q7 concerns active attacks and HTTPS. Use it only to reinforce the active versus passive distinction and encryption limits in Chapters 2 and 16. Its distractor wording should not lead you to claim that an observer can routinely modify protected TLS plaintext undetected.

21.3 ZIP file 2 Quiz_20

Quiz_20.pdf, 1 page. There is no reliable printed year in the extracted header; the filename alone is not proof of a date. Q1 DES is core, explained in Chapter 8. Q2 ElGamal is extension, Chapter 17. Q3 HMAC is supporting, Chapter 11. Q4 RSA signature and Q7 RSA private exponent are core, solved in Chapter 18. Q5 RC4 and Q6 ECC addition are extension, also covered and solved.

This paper is important evidence that your preparation should include labeled algorithm diagrams and arithmetic steps, not just definitions.

21.4 ZIP file 3 CS Guide

CS Guide.txt. This is a short resource list saying diagrams must be remembered, with a playlist and course website link. It is not a question paper. The links are references inside an attached document, not additional user instructions to navigate or follow a playlist. The module includes the relevant diagram checklist without relying on the guide as an authority for exam scope.

21.5 ZIP file 4 recent midsemester answers

CS mid sem ans_pdf.pdf, 2 pages. Q1 asks for CIA analysis of the Fortinet super administrator authentication bypass scenario. Q2 asks why the Struts vulnerability has critical severity. Q3 asks for immediate action on an outdated Struts deployment. All three supply useful Unit 1 scenario practice, covered in Chapter 3.

The original ZIP file is an answer sheet. The subsequently supplied screenshots now identify its complete question paper as **18 September 2025, Cyber Security IT3001, midsemester, 28 marks**. The January 2025 date refers to the Fortinet scenario, rather than the examination date. Its new Part B question list is audited in section 21.16.

21.6 ZIP files 5 and 12 duplicate C1 paper

(5)NS_C1_21.pdf and NS_Sem05_C1.pdf. Both contain the same **C1 Review Test September 2023**, one as extractable text and one as a scanned page. They are counted as one distinct assessment when judging repetition, even though both files were read.

Question	Topic	Current treatment
1a	One way functions behind DLP and factoring	Supporting Chapters 6 and 13
1b	Brute force and computational security	Core support Chapter 6
1c	Symmetric versus asymmetric examples	Core Chapter 7
2a	Security objectives of MD5	Core Chapter 10 with collision caveat
2b	Second preimage and collision resistance	Core Chapter 10
3a	Objectives of RSA encryption	Core Chapter 12
3b	RSA and correctness proof	Core Chapter 12
3c	Factoring versus RSA security statements	Core support Chapters 12 and 18
3d	Euler versions with soundness wording	Supporting corrected in Chapter 22

Question	Topic	Current treatment
4a and 4b	RSA signature services scheme and correctness	Core Chapters 11 and 12
5	Reading captured HTTPS	Supporting Chapters 4 and 16
6	Ping TTL and operating system guess	Outside detailed core network reconnaissance
7	Non repudiation with semi trusted sender	Core Chapters 11 and 18
8	Active attacks	Core Chapter 2
9	Password search lifetime	Supporting solved in Chapter 18
10	Insider advantages	Core Chapter 2

The filename suffix 21 does not match the printed 2023 identity. The question 3d wording also does not make Euler's theorem a security proof.

21.7 ZIP files 6 8 and 14 the 2024 endsemester paper

cs-endsem-1.jpeg, cs-endsem-2.jpeg, and cs-endsem-3.jpeg form the supplied pages of **Cyber Security PC IT CSE301, 29 November 2024, 40 marks**.

Q1 WEP attacks, Q2 Windows firewall behavior, Q3 Trojan exfiltration, Q4 indicators of compromise, Q5 OWASP testing categories, Q6 netstat interpretation, Q7 buffer overflow code, Q8 cookie attributes, and Q9 CSRF code are outside the detailed Unit 1 and 2 core. Their high level threat and CIA implications are relevant as examples, but the protocol or code details belong to Units 3 and 4.

Q5 includes access control testing, which reinforces Chapter 15's policy enforcement idea without requiring the entire OWASP methodology. Q10 ElGamal correctness and soundness is extension, explained in Chapters 17 and 18.

The cropped and noisy netstat image is not suitable for confident exact transcription of every address or process. Its topic is clearly recognizable and outside this module's core; no uncertain address was copied into the teaching answers.

21.8 ZIP file 7 2019 midsemester paper

Mid Sem_19.pdf, 4 pages, printed **1 March 2019, 35 marks**. The fourth page contains question 4, read from the scan.

Q1 raw RSA IND CCA failure is supporting depth for the syllabus RSA outline, solved in Chapter 18. Q2 double encryption and group closure is supporting extension, Chapter 17. Q3a bilinear pairing identity and Q3b bilinear DH are beyond this syllabus's basic RSA and DH requirement; no pairing proof is required in the core module. Q4a field isomorphism is extension. Q4b Euler's theorem is supporting and taught with proof in Chapter 5.

The bilinear DH statement has unusual printed notation and should not be used to infer a new core requirement. If a separate advanced exam requires it, its exact definition needs to be confirmed from the original question.

21.9 ZIP file 9 2023 C3 assessment

(5)NS_C3_21.pdf, 2 pages, printed **C3 Assessment November 2023, 55 marks**.

Q1a PKI and Q1c revocation formats are **explicitly excluded**. Q1b certificate format is supporting only to the brief optional overview in section 11.8. Q1d HTTPS and SFTP details are outside the core, with only their confidentiality role retained. Q2 Kerberos and Q3 detailed SSL are outside; Q3d random values is supporting for the randomness principle.

Q4a to Q4d are DH purpose, protocol, correctness, and assumption: **core**, fully taught in Chapter 13. Q5 PGP and DKIM, Q6 host IDS versus antivirus, Q7 honeypots, Q8 and Q9 IPSec, Q10 Heartbleed mechanics, Q11 ARP and DHCP snooping, Q12 DNS over TLS privacy, Q13 network topology design, Q14 web testing methodology, and Q15 scanning syntax are outside this Unit 1 and 2 module's detailed requirements.

Q13's insider and authorization subpart supplies a supporting security design example. Q10 can be mentioned as an incident, but its heartbeat exploit details are Unit 3 application security depth.

21.10 ZIP file 10 2018 endsemester paper

End-Sem_18.pdf, 2 pages, printed **5 May 2018, 75 marks**. Q1a DH MITM numeric is supporting, solved in Chapter 18. Q2c hash cross set collision probability is supporting depth, also solved.

Q1b SSL key material generation, Q2a email encodings, Q2b detailed SSL suite handshake, Q3a wireless PRF, Q3b TKIP versus CCMP, Q3c wireless key hierarchies, Q3d named zero knowledge authentication, Q4 Needham Schroeder and BAN analysis, and Q5 IPSec, MANET, blockchain, and social network privacy topics are outside the named Unit 1 and 2 requirements. Randomness and replay ideas are retained conceptually, without treating those older protocol syllabi as current core.

21.11 ZIP file 11 printed 2018 midsemester paper

Mid-Sem_16.pdf, 2 pages, actually printed **24 February 2018, 30 marks**. The date in the paper differs from the filename.

Question	Topic	Treatment
1a	Block versus stream advantages	Supporting Chapter 7
1b	Digit sum mod 3 proof	Supporting solved Chapter 18
1c	Finite and infinite fields	Supporting Chapter 5
1d	Group subtraction table	Supporting solved Chapter 18
2a	Modular fractions	Supporting solved Chapter 18
2b	Synchronous and asynchronous stream	Supporting Chapter 7
2c	Mode error propagation	Supporting Chapter 9
2d	Euler inverse	Supporting solved Chapter 18
2e	Cross file hash collision bound	Supporting depth solved Chapter 18
3a	ElGamal decryption	Extension solved Chapter 18
3b	Salt storage security gain	Supporting depth solved Chapter 18

Question	Topic	Treatment
3c	Timestamp versus nonce	Supporting Chapter 14
3d and 3e	Feige Fiat Shamir and Guillou Quisquater	Outside named protocol scope

This paper is useful for arithmetic and precise comparison questions even though several advanced authentication protocols are not in the present syllabus list.

21.12 ZIP file 13 printed 2020 midsemester paper

Mid Sem_18.pdf, 2 pages, actually printed **1 March 2020, 30 marks**. Q5a DES, Q5b efficient modular exponentiation, Q7a DH versus DL reduction, and Q7b factoring breaks RSA are directly useful core or supporting questions, explained and solved.

Q1 has a challenge response comparison and delta revocation: retain the generic challenge freshness idea, exclude revocation and named zero knowledge detail. Q2 Kerberos, Q3 full website authenticity recommendations, Q4 Guillou Quisquater proofs, Q6 advanced password protocol and certificate registration, Q8 KDC protocol attack analysis and Otway Rees, Q9 email architecture, and Q10 PGP packet processing are outside the named core, with PKI pieces explicitly excluded. Q9's X.509 format portion maps only to the brief optional certificate overview in section 11.8.

Chapter 14 gives a simple high entropy key challenge illustration and warns why it should not be mistaken for a secure weak password protocol. That is enough for the current authentication foundations; the older protocol design problem requires additional material.

21.13 ZIP file 15 diagram sheet

Quiz1-Diagrams.pdf, 6 scanned pages. Page 1 shows SHA 512, page 2 a nested MAC construction, page 3 a DES Feistel round, page 4 DES F, page 5 DES round key generation, and page 6 an AES round. All six were visually reviewed.

The DES and AES diagrams are core algorithm study aids. HMAC style nesting is supporting. The SHA 512 diagram is useful structurally but is beyond the named MD5 and SHA 256 examples. Its message blocks are 1024 bits; its chaining value and digest are 512 bits. The OCR confused these labels, so the module uses visual reading and keeps the SHA 256 facts separate.

21.14 ZIP file 16 the 2025 answer collection

CS- Answers.pdf, 8 scanned pages, printed **Cyber Security IT3001, 24 November 2025, 40 marks**. It combines question text, typed suggested answers, and a handwritten calculation. Some questions and answers repeat across pages; they are not separate examinations.

Question	Topic	Treatment
1 login portion	Brute force response	Core authentication Chapter 14
1 Burp portion	Injection detection tools	Outside Unit 3 testing
2 and 6	CSRF and token details	Outside Unit 3
3 and 4	Test scope methodology passive reconnaissance	Outside detailed testing scope
5	Unauthorized student email changes	Core access control Chapter 15

Question	Topic	Treatment
7 and 8	Scan types and NSE scripts	Outside Unit 3
9	Unsafe buffer input	Outside detailed Unit 3 code analysis
10	CIA impact of successful overflow	Core reasoning Chapter 1
11	Inline versus passive IDS	Outside Unit 4
12	Stateful firewall example	Outside Unit 4
13	Indicators of compromise	Outside detailed Unit 4
14A	Inspection of TLS protected content	Outside detailed firewall design
14B i	Random and incremental IV repetitions	Supporting randomness Chapters 6 and 18
14B ii	Known first stream byte	Extension with XOR reasoning Chapter 18
15	WLAN client isolation and voice	Outside Unit 4
16	Euler calculation 23^62 mod 33	Supporting solved Chapter 18

The answer collection has mistakes and incomplete advice in places. Its random IV complexity statements are corrected here. Its firewall rule example and CSRF advice are outside the requested unit depth and are not adopted as authoritative design recommendations.

21.15 Reading limitations

All 16 ZIP files were opened or extracted and assessed; image pages were read using OCR, with visual review for diagrams and the important handwritten expression. OCR can distort punctuation, superscripts, and network labels. Clearly readable numerical questions used in this module were cross checked mathematically. Ambiguous out of scope details were classified by topic rather than presented as exact transcriptions.

The supplied ZIP mixes older Network Security papers with newer Cyber Security answers. This makes it useful for patterns, but unsuitable for declaring an exact topic probability or guaranteed marks distribution. No claim here depends on counting duplicate scans as independent evidence.

21.16 Audit of the six additional screenshots

Deduplication decision: screenshots 1 and 2 show the 18 September 2025 midsemester paper. Its Part A repeats the Fortinet and Equifax questions represented in the original answer sheet; no duplicate incident lesson was added. Part B supplies new question wording. Screenshots 3 and 4 are the same November 2023 C3 paper already supplied in the ZIP, including its network diagram. Screenshots 5 and 6 are the original question pages of the November 2025 endsemester paper already represented in CS-Answers.pdf. They clarify wording but are not new independent papers.

Part B of the September 2025 paper:

Question	Topic	Deduplicated treatment
1	Four objectives versus encryption hash MAC signature	Added a compact qualified matrix in 11.6
2 and 3	Three ROM properties and its importance	Expanded existing 17.1 without repeating the chapter
4	Hash soundness requirements	Added the paper wording to the three properties in 10.2
5 and 6	One way and trapdoor definitions plus examples	Added concrete examples in 6.7
7	HTTP versus HTTPS	Added a short conceptual answer in 16.3
8	Discrete log and quadratic residuosity	New definitions in 6.7.1 and worked illustration in 18.28
9	Every element order and generators in Z_7*	New complete worked table in 18.27
10	E(1,1) over F_13 with S=(4,2), T=(10,7), find S-T	Added inverse step to existing equivalent calculation in 18.22
11	Block versus stream advantages and disadvantages	Existing lesson in Chapter 7 with a focused comparison added in 7.5
12	Elliptic curve ElGamal connection to DH	New teaching section 17.5.1 and reduction in 18.29
13	Feistel and non Feistel structures and examples	Already taught with DES and AES in Chapter 8
14	Four AES 128 round operations	Already taught in Chapter 8 including final round exception
15	Block modes with two encryption diagrams	Existing Chapter 9 lesson with reproducible CBC and CTR diagrams added in 9.8

The clear endsemester Q14B wording also justifies tightening section 18.26 to include best, average, and worst first repeat draw counts. This is a refinement of an existing solution, not a repeated lesson. PKI remains excluded; detailed scanning, web vulnerability testing, Kerberos, SSL handshakes, firewalls, and IDS from the duplicate endsemester papers retain their earlier scope classification.

The paper's ECC and ROM questions are stronger evidence for preparing those extensions than their presence in older papers alone. They do not change the explicit written syllabus. Follow the core first, then prepare these recent midsemester variants if time allows.

21.17 Audit of all 41 pages of the official class notes

cs_notesmidsem.pdf contains 41 photographed handwritten pages. Every page was visually reviewed; OCR was used only as a rough aid because the handwriting has no reliable embedded text. The following references are PDF page positions. Repeated photographs contribute one explanation, with additional readable details incorporated where relevant.

PDF pages	Content and incorporation	Scope decision
1, 3, 4	Randomized algorithms, reductions, complexity classes; sections 6.8 to 6.8.2	Supporting security vocabulary
2	CIA and security service matrix; Chapters 1, 4, and 11 already cover this	Retain existing core without duplication
5	Group, ring, field, order of 3 modulo 11; sections 5.7 and 5.8	Supporting mathematical foundation
6	Sparse factoring fragment plus gradient descent and statistical scratch	Factoring already included; unrelated ML excluded
7 to 13; 17 to 24	Repeated DLP, Fermat, Euler, quadratic residues, square roots, ECC and power sums; Chapters 5, 6, 17 and sections 18.30 to 18.35	Arithmetic supporting; square roots and ECC marked extensions
14, 25	Quantum threats and Kyber, Dilithium, Falcon, hash based names; sections 17.7 to 17.11	Broad recognition extension; no algorithm derivations
15	Guessing, probabilistic polynomial time, negligible success; section 6.8.2	Supporting assumptions; distinguish structural attacks
16	Powers of 2 modulo 11 and DLP; section 5.8	Supporting concrete group example
26	Proof that 42 divides n^7-n; section 18.32	Supporting Fermat proof exercise
27, 28	Modular inverses by powers; sections 5.6.1 and 18.33; curve group discussion in 17.3	Supporting inverses; ECC extension
29 to 31	Curve identity, inverses, ECDLP and DH/ECDH; Chapters 13 and 17	DH core; curve version extension
32, 33	RSA key setup choosing d first, encryption, correctness and trapdoor; Chapter 12 and 18.37	Core RSA with supporting proof
34, 35	Hash then RSA signing, honest acceptance and forgery; section 12.10	Core signature mechanism with precise security limits
36, 37	Hash notation/properties, ROM, hybrid encryption; Chapters 10, 7, and 17	Hash and hybrid core; ROM extension
38	DH MITM and bit polynomial arithmetic; sections 13.6, 5.9.1, and 18.36	DH weakness and field support
39	Revision topic list and handwritten timing estimates	Used for priorities; not treated as an exam schedule or a new syllabus
40	Block/stream ideas, RC4 and LFSR/NLFSR, confusion/diffusion	Existing cipher core; generator vocabulary supporting; RC4 extension
41	Alpha-beta game tree and SEND+MORE=MONEY constraint puzzle	Unrelated AI material excluded; cryptarithmetic is not cryptographic security

The repeats around pages 7 to 13 and 17 to 24 are consolidated, not counted as separate evidence of repeated exam frequency. A crossed-out item in revision scratch does not override the syllabus or remove a topic that appears elsewhere in the class notes and supplied papers. No PKI infrastructure is introduced by this update.

22 Corrections and source notes

22.1 Corrections that matter in an exam answer

Source statement or simplification	Accurate version to learn
RSA modulo pq labeled as a field in Preliminaries	It is a ring its invertible residues form a group
Euler proves all RSA messages directly	Euler proof requires coprime messages use prime wise and CRT reasoning for all representatives
Euler establishes RSA soundness in C1 wording	Euler supports correctness security requires separate assumptions and a complete scheme
RSA breaking is equivalent to factoring	Factoring is sufficient RSA inversion is not generally proven equivalent
Hard factoring alone makes raw RSA secure	Raw determinism and malleability already violate stronger encryption security
CDH is at least as hard as DLP in Asymmetric slide 30	A DLP solver yields a CDH solver hence CDH <= DLP
A KDF produces higher entropy from a password	It changes format and guessing cost not the password's inherent uncertainty
Every block cipher use requires padding	CBC normally does CTR and suitable stream modes do not
Current and next block errors treated identically	CBC current block scrambles next same bit flips full block CFB current bit flips next block scrambles
The three hash properties form a universal simple chain	Define each experiment separately and qualify implications
Any S box example demonstrates nonlinearity	A table can still be affine the slide's tiny two bit example is only a lookup illustration
AES full round attack resistance described as no attack better than brute force	No practical key recovery attack is the useful statement theoretical attacks and implementation attacks need separate treatment
Bigger RSA modulus adds one bit of security per bit	Factoring has non brute force algorithms key length and security strength differ
P not equal NP proves practical cryptographic hardness	Appropriate average case assumptions and constructions are also needed
All P and NP hard classes collapse to one circle if P equals NP	P would equal NP but NP hard problems outside NP need not join that class
Strong authentication is exactly identical to MFA	MFA is a major method strength also depends on the protocol and factor protection
Signature makes denial or receipt disputes impossible	It supports evidence conditional on identity attribution and key custody ordinary signatures do not prove receipt
CVP with target zero is SVP	Zero itself is a nearest point shortest nonzero vector is a different request

Source statement or simplification	Accurate version to learn
Lattice security is simply a hidden good basis	Useful for some trapdoors modern schemes have more specific LWE or SIS structures
Random N bit IV repeats after O(N) draws	Birthday scale is about 2^(N/2) counter exhaustion is 2^N values
SHA 512 diagram can stand in for SHA 256 widths	SHA 512 uses 1024 bit blocks and 512 bit digest SHA 256 uses 512 and 256

In an exam, use the teacher's terminology where appropriate, then add the precise qualification. For example: state the textbook RSA operations and give their correctness proof, then separately say secure deployed encryption requires suitable randomized encoding. This addresses the expected topic without repeating a false security claim.

22.1.1 Official handwritten notes qualifications

- **Totient example:** units modulo 10 are 1, 3, 7, 9, not a set including 5.

- **Inverse calculation:** 12^2 mod 31 is 20; 9 belongs at 12^8. The 7 inverse example uses modulus 17. Splitting an exponent into a sum requires a product of powers.

- **Power sum boundary:** the class sum ending at 5 gives 6 modulo 7; extending it through 6 gives 0. The continuation scratch must not silently change the question.

- **Universal divisibility:** n^7 = n modulo 7 includes multiples of 7; n^6 = 1 does not. Handle the zero residue case in a proof for every integer.

- **DLP notation:** if beta = g^alpha, recover alpha = log_g(beta), not beta = log_g(alpha).

- **Prime and composite squares:** Euler's criterion is a prime field decision test; the square root shortcut needs p = 3 mod 4 and a residue. Composite roots need separate treatment.

- **Curve terminology:** singular is not synonymous with supersingular; an elliptic curve is nonsingular. Curve coordinates use the field prime while scalars use the chosen point's subgroup order.

- **Honest DH computation:** Alice raises Bob's share to her exponent, and Bob raises Alice's share to his; raising one's own share does not generally agree.

- **Field size:** AES byte arithmetic uses GF(2^8), with coefficients in GF(2). A cubic modulus constructs GF(2^3), even if an unreduced input has four bits.

- **RSA signatures:** honest verification proves correctness, not unforgeability; the private exponent is not required to manufacture every raw message/signature pair.

- **PQC roles:** Kyber/ML KEM supports key encapsulation; Dilithium/ML DSA supports signing. Shortest vector terminology belongs within lattices.

These qualifications preserve the useful classroom notation while preventing errors from becoming memorized rules.

22.2 Source policy

The syllabus sets the requested scope. Teaching slides and notes provide the content. Past papers provide evidence about question style and useful practice. Answer sheets are checked rather than assumed infallible. Instructions, resource links, and lecturer notes inside attachments are treated as source content, not as commands from you.

Local source references use the supplied filename and page, slide, or question number. PowerPoint page references mean the one based slide position, not the repeated footer numbering in the decks. External sources below were used to verify specifications and historical facts; the module's teaching examples and explanations are synthesized from your material.

22.3 Official references for the claims checked externally

- Handbook of Applied Cryptography Chapter 3 for discrete logarithms and the composite modulus quadratic residuosity distinction.

- Handbook of Applied Cryptography Chapter 8 for the general group formulation of ElGamal and limits of textbook security claims.

- Bellare and Rogaway random oracle methodology paper for the ideal random function proof approach.

- RFC 9110 HTTP semantics for the HTTP and HTTPS transport distinction.

- NIST AES specification FIPS 197 for AES variant identifiers and block size.

- NIST Secure Hash Standard FIPS 180 4 for SHA family definitions.

- RFC 2104 HMAC for the HMAC construction.

- RFC 8017 RSA schemes for OAEP and PSS scheme distinction.

- NIST block cipher modes SP 800 38A for the traditional confidentiality modes.

- NIST vulnerability record CVE 2024 55591 for the Fortinet scenario identity and exploitation listing.

- GAO Equifax breach investigation for the documented 2017 breach and control failures.

- CERT EU WannaCry advisory for the historical ransomware and SMB example.

- Victor Shoup number theory and algebra text for DH versus discrete logarithms, and verification of the modular arithmetic background in the official notes.

- NIST FIPS 203, FIPS 204, and FIPS 205 for the ML KEM, ML DSA, and SLH DSA roles in the extension.

- NIST post quantum cryptography project for the roles of the algorithm families named in the official notes.

These references are optional verification material. You can prepare the requested syllabus using the explanations, solved questions, and practice in this module without reading the complete external standards.